

CFD Simulation of Compressible Flow Past a Bump Using Finite Volume Techniques

Purna Ananthkrishnan (AE22B003), Yash Shajil (AE22B110)

Department of Aerospace Engineering, Indian Institute of Technology Madras

Abstract: Compressible Euler equations are solved using an explicit finite-volume method with Van Leer, AUSM, and Roe flux schemes for interface flux evaluation. The formulation is applied to flow over a bump for freestream Mach numbers $M_\infty = 0.4, 0.8, 1.8$ with a gauge pressure of $p_\infty = 0.01$ atm. A first-order explicit time-marching scheme is used to obtain the steady-state solution, and is then extended to a higher resolution using Heun's RK2 method. Spatial accuracy is improved from first-order to higher-order reconstruction using limiter functions such as minmod, Van Leer, and Superbee.

1 Introduction

The compressible Euler equations are expressed in conservation form (equation 1) to be implemented in a finite-volume scheme:

$$\frac{\partial \bar{q}}{\partial t} + \nabla \cdot \bar{f} = 0, \quad (1)$$

where the vector of conservative variables $\bar{q}$ is

$$\bar{q} = \begin{pmatrix} \rho \\ \rho u \\ \rho v \\ \rho e_T \end{pmatrix},$$

and $\bar{f}$ denotes the corresponding flux vector. The same system can be rewritten in terms of the primitive variables $\bar{v}$ as

$$\frac{\partial \bar{v}}{\partial t} + A_{\bar{v}} \frac{\partial \bar{v}}{\partial x} = 0, \quad (2)$$

where the primitive variable vector $\bar{v}$ is

$$\bar{v} = \begin{pmatrix} \rho \\ u \\ v \\ p \end{pmatrix},$$

and $A_{\bar{v}}$ is the Jacobian matrix of the flux with respect to the primitive variables.

2 Computational Domain

The scheme is used to simulate flow over a bump geometry. A structured curvilinear mesh is used over the domain as given in figure 1, with 97 grid points along the x-direction and 49 grid points along the y-direction.

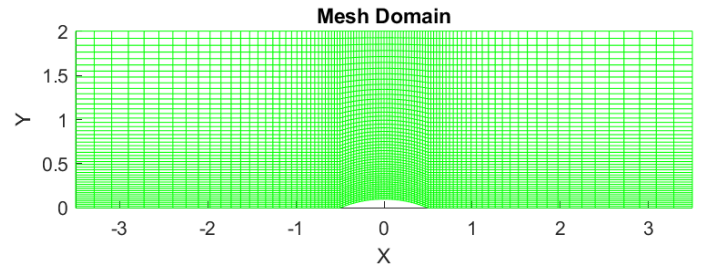


Fig. 1: Meshing of the Computational Domain

3 Numerical Method

3.1 Boundary Conditions

The boundary conditions at the inflow were modelled as the freestream conditions for velocities and density. The pressure was modelled as p_∞ for supersonic inflows and a zero gradient condition on pressure was applied for subsonic inflows. At the outflow, zero gradient conditions were applied for the velocities and density. Pressure was set to p_∞ as back pressure for subsonic outflow and zero pressure gradient was applied for supersonic outflow. The top and bottom boundaries are taken to be symmetric slip walls, with zero convection of flux and zero pressure and density gradients.

3.2 Interface Fluxes

The fluxes at the interface are found using Roe Scheme, Van-Leer Scheme and AUSM scheme.

3.2.1 Roe Scheme: The interface flux using Roe flux differencing scheme is written as follows. For a face with left and right primitive states

$$(\rho_l, u_l, v_l, p_l), \quad (\rho_r, u_r, v_r, p_r),$$

area A and outward unit normal $\mathbf{n} = (n_x, n_y)$, the Roe flux in the normal direction can be written as

$$\mathbf{F}_{\text{Roe}} = \frac{1}{2} (\mathbf{F}_l + \mathbf{F}_r) - \frac{1}{2} \mathbf{d}, \quad (3)$$

where $\mathbf{F}_l$ and $\mathbf{F}_r$ are the physical fluxes built from the left and right states and $\mathbf{d}$ is the Roe dissipation vector.

The Roe-averaged quantities are constructed as

$$r = \sqrt{\frac{\rho_R}{\rho_L}}, \quad (4)$$

$$\rho_R = \sqrt{\rho_L \rho_R}, \quad (5)$$

$$u_R = \frac{u_L + r u_R}{1 + r}, \quad (6)$$

$$v_R = \frac{v_L + r v_R}{1 + r}, \quad (7)$$

$$h_{0,R} = \frac{h_{0,l} + r h_{0,r}}{1 + r}, \quad (8)$$

with Roe-averaged sound speed and contravariant velocity

$$a_R^2 = (\gamma - 1) \left(h_{0,R} - \frac{1}{2} (u_R^2 + v_R^2) \right), \quad (9)$$

$$v_{n,R} = u_R n_x + v_R n_y. \quad (10)$$

The jumps across the face are

$$\Delta \rho = \rho_r - \rho_l, \quad (11)$$

$$\Delta u = u_r - u_l, \quad (12)$$

$$\Delta v = v_r - v_l, \quad (13)$$

$$\Delta p = p_r - p_l, \quad (14)$$

$$\Delta v_n = (u_r n_x + v_r n_y) - (u_l n_x + v_l n_y). \quad (15)$$

The intermediate coefficients are

$$\alpha_1 = A |v_{n,R}| \left(\Delta \rho - \frac{\Delta p}{a_R^2} \right), \quad (16)$$

$$\alpha_2 = \frac{A}{2a_R^2} |v_{n,R} + a_R| (\Delta p + \rho_R a_R \Delta v_n), \quad (17)$$

$$\alpha_3 = \frac{A}{2a_R^2} |v_{n,R} - a_R| (\Delta p - \rho_R a_R \Delta v_n), \quad (18)$$

$$\alpha_4 = \alpha_1 + \alpha_2 + \alpha_3, \quad (19)$$

$$\alpha_5 = a_R (\alpha_2 - \alpha_3), \quad (20)$$

$$\alpha_6 = A |v_{n,R}| (\rho_R \Delta u - n_x \rho_R \Delta v_n), \quad (21)$$

$$\alpha_7 = A |v_{n,R}| (\rho_R \Delta v - n_y \rho_R \Delta v_n). \quad (22)$$

The components of the dissipation vector $\mathbf{d}$ are then

$$d_1 = \alpha_4, \quad (23)$$

$$d_2 = u_R \alpha_4 + n_x \alpha_5 + \alpha_6, \quad (24)$$

$$d_3 = v_R \alpha_4 + n_y \alpha_5 + \alpha_7, \quad (25)$$

$$d_4 = h_{0,R} \alpha_4 + v_{n,R} \alpha_5 + u_R \alpha_6 + v_R \alpha_7 - \frac{a_R^2}{\gamma - 1} \alpha_1. \quad (26)$$

The mass flux contributions from left and right states are

$$v_{n,l} = u_l n_x + v_l n_y, \quad v_{n,R} = u_r n_x + v_r n_y, \quad (27)$$

$$f_m^+ = \frac{1}{2} A \rho_l v_{n,l}, \quad f_m^- = \frac{1}{2} A \rho_r v_{n,r}, \quad (28)$$

and the flux components can be written as

$$F_1 = f_m^+ + f_m^- - \frac{1}{2} d_1, \quad (29)$$

$$F_2 = f_m^+ u_l + f_m^- u_r + \frac{1}{2} (p_l + p_r) A n_x - \frac{1}{2} d_2, \quad (30)$$

$$F_3 = f_m^+ v_l + f_m^- v_r + \frac{1}{2} (p_l + p_r) A n_y - \frac{1}{2} d_3, \quad (31)$$

$$F_4 = f_m^+ h_{0,l} + f_m^- h_{0,r} - \frac{1}{2} d_4. \quad (32)$$

3.2.2 Van - Leer Scheme: The 2-D formulation of the Van-Leer interface flux can be written as

$$\begin{aligned} \tilde{F}_{i+\frac{1}{2}} = A_{i+\frac{1}{2}} & \left[\rho_i C_{VL}^+ a_i \begin{pmatrix} 1 \\ u \\ v \\ h_0 \end{pmatrix}_i + \rho_{i+1} C_{VL}^- a_{i+1} \begin{pmatrix} 1 \\ u \\ v \\ h_0 \end{pmatrix}_{i+1} \right] \\ & + (\tilde{D}_i p_i + \tilde{D}_{i+1} p_{i+1}) \begin{pmatrix} 0 \\ n_x \\ n_y \\ 0 \end{pmatrix}. \end{aligned} \quad (33)$$

The Van-Leer mass flux coefficients are

$$C_{VL}^+ = \alpha_i^+ (1 + \beta_i) M_i - \frac{\beta_i}{4} (M_i + 1)^2, \quad (34)$$

$$C_{VL}^- = \alpha_{i+1}^- (1 + \beta_{i+1}) M_{i+1} + \frac{\beta_{i+1}}{4} (M_{i+1} - 1)^2, \quad (35)$$

$$\tilde{D}_i = \alpha_i^+ (1 + \beta_i) - \beta_i D_i^+, \quad (36)$$

$$\tilde{D}_{i+1} = \alpha_{i+1}^- (1 + \beta_{i+1}) - \beta_{i+1} D_{i+1}^-, \quad (37)$$

$$D_i^+ = \frac{1}{4} (M_i + 1)^2 (2 - M_i), \quad (38)$$

$$D_{i+1}^- = \frac{1}{4} (M_{i+1} - 1)^2 (2 + M_{i+1}) \quad (39)$$

α_i^+ and α_{i+1}^- are found as

$$\alpha_i^+ = \frac{1}{2} (1 + \text{sgn}(1.0, M_i)), \quad (40)$$

$$\alpha_{i+1}^- = \frac{1}{2} (1 - \text{sgn}(1.0, M_{i+1})), \quad (41)$$

where

$$\text{sgn}(1.0, M) = \begin{cases} 1, & M > 0, \\ -1, & M < 0. \end{cases}$$

And, β_i and β_{i+1} are found as

$$\beta_i = -\max[0.0, 1.0 - \text{int}(|M_i|)], \quad (42)$$

$$\beta_{i+1} = -\max[0.0, 1.0 - \text{int}(|M_{i+1}|)]. \quad (43)$$

3.2.3 AUSM Scheme: The Advection Upstream Splitting Method (AUSM) is a method similar in principle to the Van Leer flux splitting, and the interface flux is found as

$$\begin{aligned} \tilde{F}_{i+\frac{1}{2}} = A_{i+\frac{1}{2}} & \left[\rho_i C_{LS}^+ a_i \begin{pmatrix} 1 \\ u \\ v \\ h_0 \end{pmatrix}_i + \rho_{i+1} C_{LS}^- a_{i+1} \begin{pmatrix} 1 \\ u \\ v \\ h_0 \end{pmatrix}_{i+1} \right] \\ & + (\tilde{D}_i p_i + \tilde{D}_{i+1} p_{i+1}) \begin{pmatrix} 0 \\ n_x \\ n_y \\ 0 \end{pmatrix}. \end{aligned} \quad (44)$$

Where, C_{LS}^+ and C_{LS}^- are found as,

$$C_{LS}^+ = \max[0.0, C_{VL}^+ + C_{VL}^-], \quad (45)$$

$$C_{LS}^- = \min[0.0, C_{VL}^+ + C_{VL}^-] \quad (46)$$

3.3 Residual Calculation

The residual for a given time-step (or a given iteration) is found using the interface fluxes as

$$\bar{R}_{i,j}^n = \bar{F}_i^n - \bar{F}_{i-1}^n + \bar{G}_j^n - \bar{G}_{j-1}^n \quad (47)$$

The residuals obtained are used to update the values of the flow variables at each time step and to check for convergence of the scheme.

3.4 Time Marching

The flow variables (p, u, v) are computed for the subsequent $(n+1)^{th}$ time step using the residuals

$$\bar{q}_{i,j}^{n+1} = \bar{q}_{i,j}^n - \frac{\Delta t_{i,j}}{V_{i,j}} \bar{R}_{i,j}^n \quad (48)$$

The time steps are calculated for each cell individually as

$$\frac{\Delta t_{i,j}}{V_{i,j}} = \frac{CFL}{\sum_{k=1}^4 A_k |\lambda_k|} \quad (49)$$

where, k denotes the faces of each cell and the corresponding maximum characteristic wave speed, λ_k , which is $U + a$, where U is the contravariant velocity corresponding to the particular face.

3.5 Convergence

The residuals are non-dimensionalised by appropriate scales and the norm of this non-dimensional residual is monitored to converge to a specified tolerance. Thus the convergence criterion used is

$$\left| \frac{\begin{bmatrix} \frac{1}{\rho u_\infty} & 0 & 0 & 0 \\ 0 & \frac{1}{\rho u_\infty^2} & 0 & 0 \\ 0 & 0 & \frac{1}{\rho u_\infty^2} & 0 \\ 0 & 0 & 0 & \frac{1}{\rho u_\infty h_\infty} \end{bmatrix} \bar{R}_{i,j}^n}{\begin{bmatrix} \frac{1}{\rho u_\infty} & 0 & 0 & 0 \\ 0 & \frac{1}{\rho u_\infty^2} & 0 & 0 \\ 0 & 0 & \frac{1}{\rho u_\infty^2} & 0 \\ 0 & 0 & 0 & \frac{1}{\rho u_\infty h_\infty} \end{bmatrix} \bar{R}_{i,j}^0} \right| < \epsilon$$

3.6 Higher Order Spatial Reconstruction

At the interface between the i^{th} and the $(i+1)^{th}$ cells $(i+1/2)$, the all the relevant quantities, generalised as ϕ at the left and right is reconstructed as

$$\phi_{i+\frac{1}{2},L} = \phi_i + \frac{1}{2} \psi r_{i-\frac{1}{2}}^+ \delta \phi_{i-\frac{1}{2}} \quad (50)$$

$$\phi_{i+\frac{1}{2},R} = \phi_{i+1} + \frac{1}{2} \psi r_{i+\frac{3}{2}}^+ \delta \phi_{i+\frac{1}{2}} \quad (51)$$

where, $r_{i-\frac{1}{2}}^+ = \frac{\delta \phi_{i-\frac{1}{2}}}{\delta \phi_{i+\frac{1}{2}}} = \frac{1}{r_{i-\frac{1}{2}}^+}$ and ψ is a limiter function.

The limiter functions employed for higher-order spatial reconstruction are the minmod, Superbee, and Van Leer limiters.

3.7 Higher Temporal Resolution

Higher temporal resolution was incorporated using an RK2 routine based on Heun's scheme, as shown below

$$\frac{d\phi}{dt} = f(\phi, t) \quad (52)$$

$$\phi^n = \phi^{n-1} + 0.5 * (k_1 + k_2) \quad (53)$$

Where,

$$k_1 = \Delta t f(\phi^n, t^n), \quad (54)$$

$$k_2 = \Delta t f(\phi^n + k_1, t^n + \Delta t) \quad (55)$$

4 Parameters

The flow parameters taken were $T_\infty = 300$ K, and M_∞ as 0.4, 0.8 and 1.4. The gauge pressure was taken as 0.01 atmospheres. The tolerance for convergence was set at $1e-3$, and the CFL number was taken to be 0.2, except when implementing higher order schemes, for which the CFL was reduced to ensure numerical stability.

5 Results

5.1 AUSM Scheme

The results presented below were computed using the AUSM scheme under **first-order** spatial and temporal accuracy.

5.1.1 $M_\infty = 0.4$: Convergence plot decreases but with lot of oscillations.

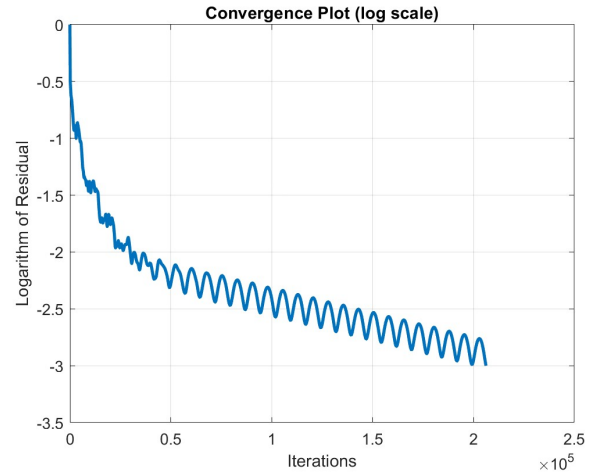


Fig. 2: Convergence Plot, AUSM Scheme, $M_\infty = 0.4$

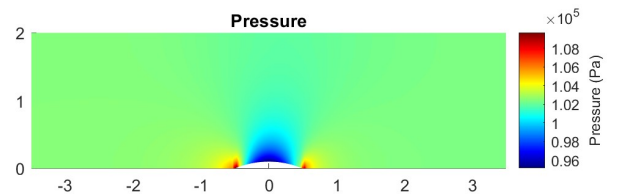


Fig. 3: Pressure Contour, AUSM Scheme, $M_\infty = 0.4$

Near the leading edge and trailing edge of the bump, the pressure high due to stagnation. Over the bump the flow accelerates and hence the pressure reduces. After the half point (center region of the bump) the flow starts decelerating.

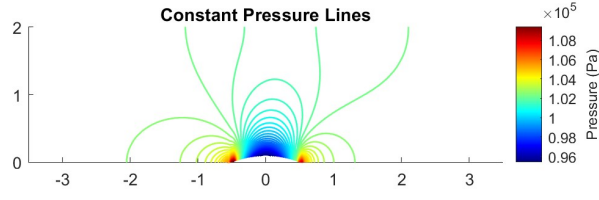


Fig. 4: Constant Pressure Lines, AUSM Scheme, $M_\infty = 0.4$

Near the bump the pressure gradients are high and hence contour lines are dense.

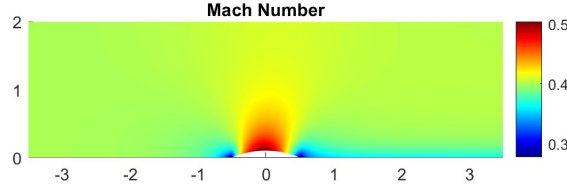


Fig. 5: Mach Number contour, AUSM Scheme, $M_\infty = 0.4$

Near the trailing edge and the leading edge, the velocities reach stagnation and hence low mach numbers. Over the bump the flow is accelerating and hence the mach number increases till the half point but does not reach sonic.

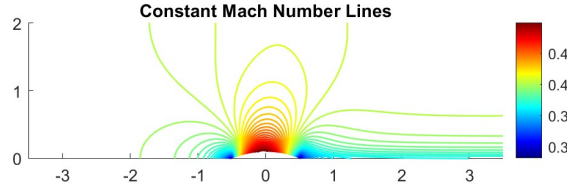


Fig. 6: Mach Number Contour Lines, AUSM Scheme, $M_\infty = 0.4$

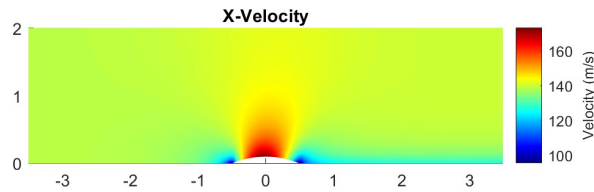


Fig. 7: X-velocity contour, AUSM Scheme, $M_\infty = 0.4$

Velocity stagnates at the leading and trailing edge of the bump and accelerates on top of the bump and then starts decelerating after the half point.

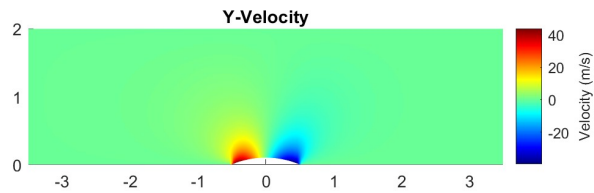


Fig. 8: Y-velocity contour, AUSM Scheme, $M_\infty = 0.4$

On the left half of the bump the velocity is in upward direction and on the right half the velocity is in downward direction. Due to

the inviscid nature of the governing equations used, there is flow separation or boundary layer separations and hence is symmetric about the centerline passing through the bump.

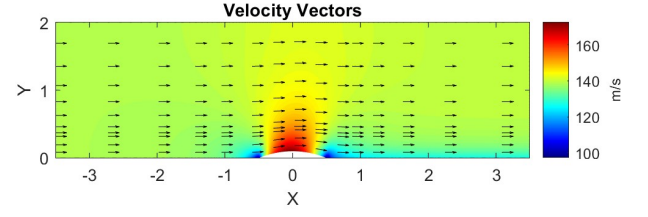


Fig. 9: Velocity Vector Plot, AUSM Scheme, $M_\infty = 0.4$

5.1.2 $M_\infty = 0.8$: The convergence graph initially decreases, then rises before decreasing again and stabilizing at the desired tolerance level. With a higher CFL, convergence occurs more rapidly; however, there is a maximum CFL value beyond which the solution fails to converge and instead diverges.

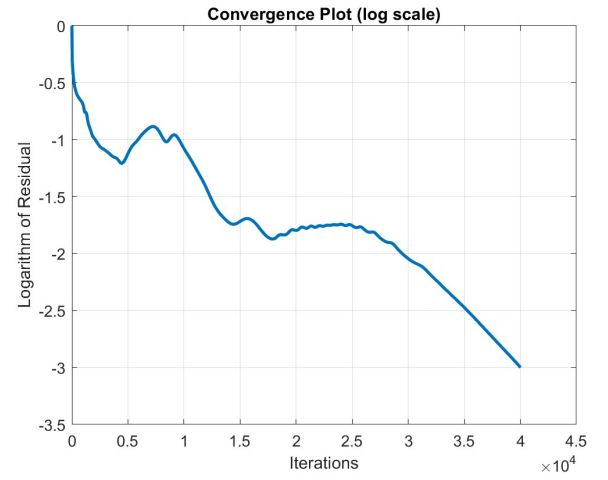


Fig. 10: Convergence Plot, AUSM Scheme, $M_\infty = 0.8$

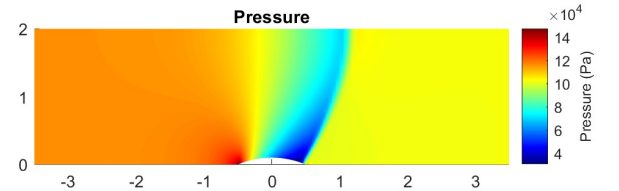


Fig. 11: Pressure Contour, AUSM Scheme, $M_\infty = 0.8$

The pressure at the front edge of the bump is at its maximum because of stagnation. As the velocity increases over the bump, the pressure decreases. The velocity achieves sonic speed approximately at the bump's center. Beyond the bump, when it meets the horizontal surface, a shock wave forms due to the supersonic flow. Following the shock, the pressure rises to high levels, as anticipated.

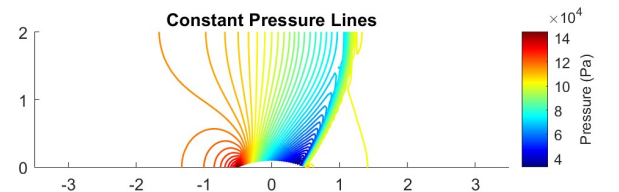


Fig. 12: Constant Pressure Lines, AUSM Scheme, $M_\infty = 0.8$

The contour lines are tightly packed around the shock area, indicating steep pressure gradients. Beyond the shock, the pressure gradients decrease significantly.

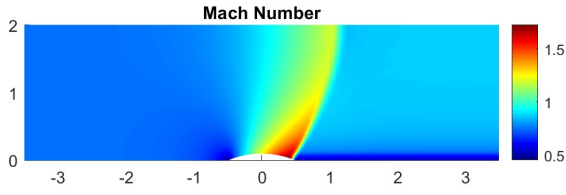


Fig. 13: Mach Number contour, AUSM Scheme, $M_\infty = 0.8$

As the flow approached the bump, the mach number increases from its initial value 0.8 to 1.5+. At the end of the bump, flow being supersonic due to change in the direction of the flow the information is not propagated upstream and hence causes discontinuity in the form of shock. Other side of the shock the mach numbers are subsonic.

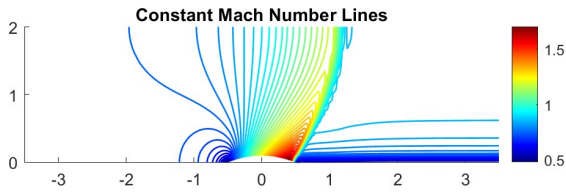


Fig. 14: Mach Number Contour Lines, AUSM Scheme, $M_\infty = 0.8$

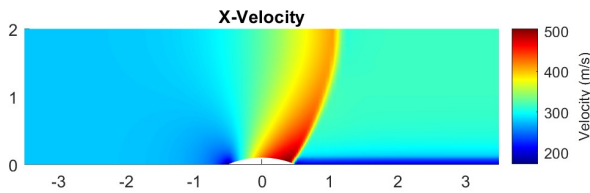


Fig. 15: X-velocity contour, AUSM Scheme, $M_\infty = 0.8$

X velocity reduces near the leading edge due to the stagnation and then increases over the bump till it reaches the shock after which there is a drop in the velocity and remains constant in the rest of the domain.

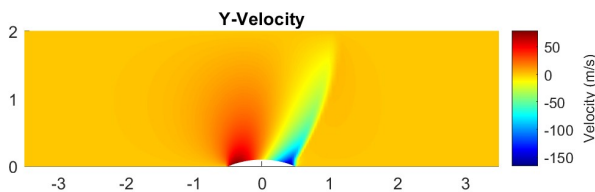


Fig. 16: Y-velocity contour, AUSM Scheme, $M_\infty = 0.8$

Y velocity initially 0 at the inlet increases in the upward direction (+ve) due to the presence of the bump and then on the other half of the bump the velocity is in the downward direction (-ve).

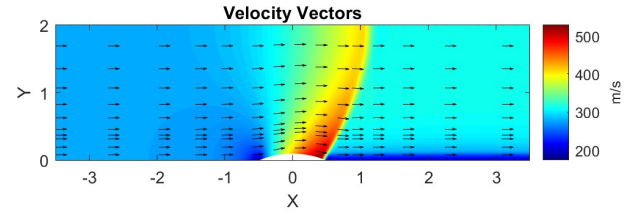


Fig. 17: Velocity Vector Plot, AUSM Scheme, $M_\infty = 0.8$

5.1.3 $M_\infty = 1.4$: Residue is decreasing for the most part with little oscillations.

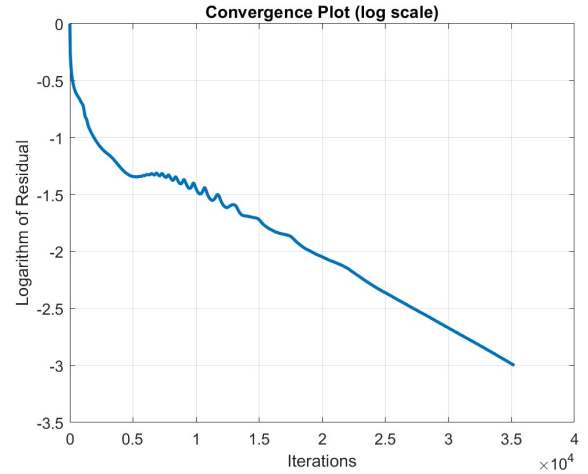


Fig. 18: Convergence Plot, AUSM Scheme, $M_\infty = 1.4$

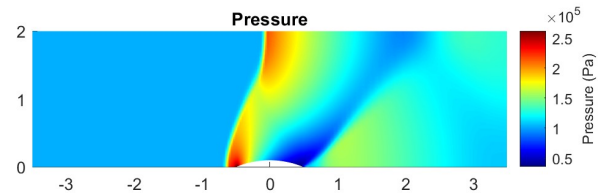


Fig. 19: Pressure Contour, AUSM Scheme, $M_\infty = 1.4$

Since the flow is supersonic, there is a shock formation upstream of the bump. The shape of the shock is like a bow. Downstream of the shock the pressure reaches high values. There is a shock which is coming from the top of the domain. This might be due to reflection of the bow shock. The flow is getting pushed up due to the presence of the bump. But since we have imposed symmetric conditions on the wall, the flux through the upper wall is 0 and hence might be the reason for this discontinuity. Downstream of the bump, the pressure increases again due to presence of another shock.

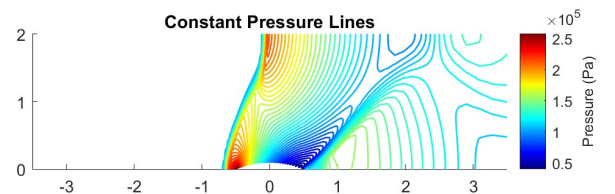


Fig. 20: Constant Pressure Lines, AUSM Scheme, $M_\infty = 1.4$

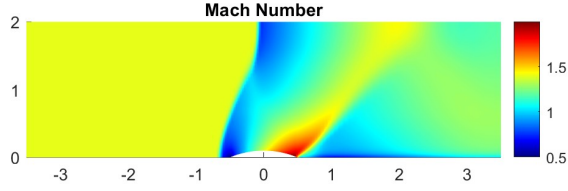


Fig. 21: Mach Number contour, AUSM Scheme, $M_\infty = 1.4$

There is drop in the mach number across the shock as expected. The mach number after the shock is around 0.7 which matches with the gas table values. Near the bump the flow again accelerates reach sonic speed before centerline and ending up with supersonic speed at the trailing edge leading to another shock due to change in direction of the flow. This again drops the mach number. Unlike the previous shock, the column of the air which becomes supersonic is less and hence the shock is also becomes weaker away from the bump.

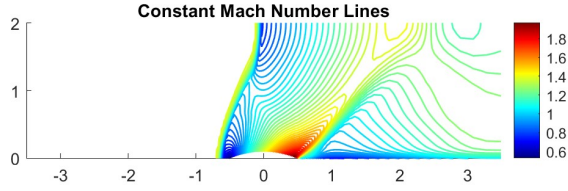


Fig. 22: Mach Number Contour Lines, AUSM Scheme, $M_\infty = 1.4$

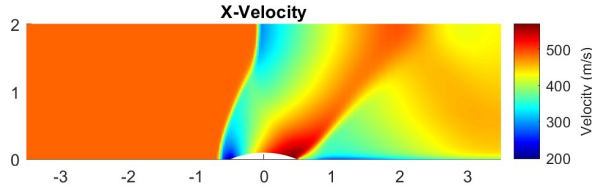


Fig. 23: X-velocity contour, AUSM Scheme, $M_\infty = 1.4$

Across the shock, the velocity drops. Over the bump the flow accelerates reaching super sonic velocity at the trailing edge of the bump. There it encounters another shock and hence reduces the velocity.

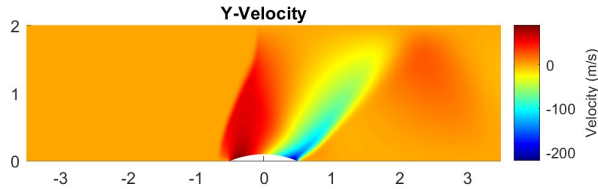


Fig. 24: Y-velocity contour, AUSM Scheme, $M_\infty = 1.4$

Y velocity is initially zero, then downstream of the shock there is an upward velocity. After the half point on the bump the velocity becomes negative and reaches high values till it encounters the second shock.

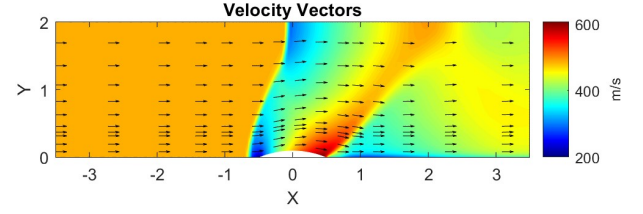


Fig. 25: Velocity Vector Plot, AUSM Scheme, $M_\infty = 1.4$

5.2 Roe Scheme

The results presented below were computed using Roe's scheme under **second-order** spatial and temporal accuracy using **minmod** limiter and **Huen's scheme** respectively.

5.2.1 $M_\infty = 0.4$: The convergence plot shows a lot of oscillations, this would probably be caused by the high CFL value set for faster convergence. The results obtained are more-or-less similar to that obtained from AUSM scheme the velocity gradients near the bump seem sharper, which could be attributed to higher spatial accuracy.

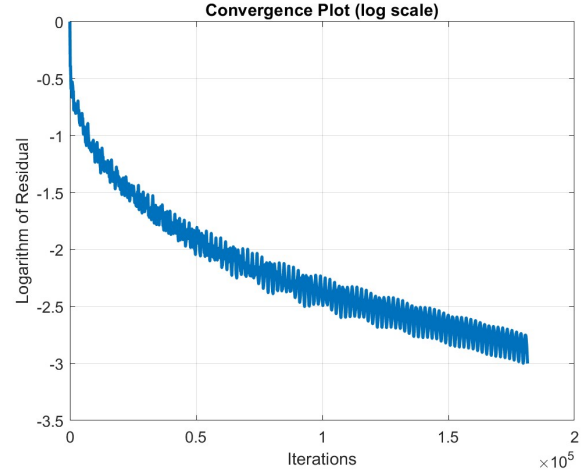


Fig. 26: Convergence Plot, Roe Scheme, $M_\infty = 0.4$

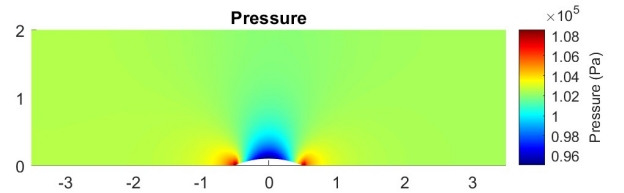


Fig. 27: Pressure Contour, Roe Scheme, $M_\infty = 0.4$

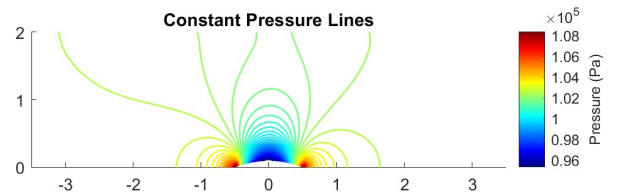


Fig. 28: Constant Pressure Lines, Roe Scheme, $M_\infty = 0.4$

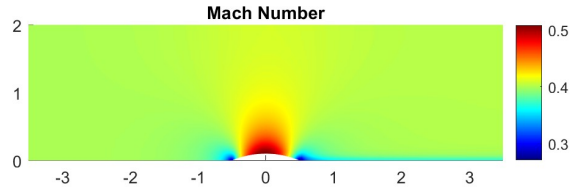


Fig. 29: Mach Number contour, Roe Scheme, $M_\infty = 0.4$

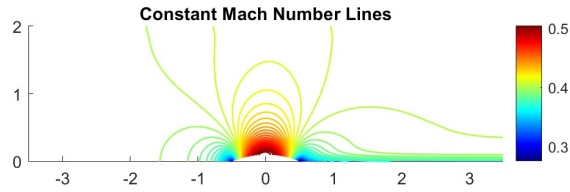


Fig. 30: Mach Number Contour Lines, Roe Scheme, $M_\infty = 0.4$

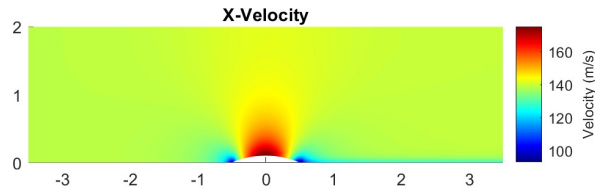


Fig. 31: X-velocity contour, Roe Scheme, $M_\infty = 0.4$

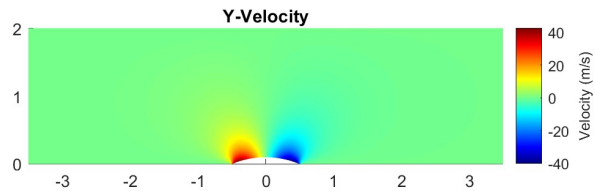


Fig. 32: Y-velocity contour, Roe Scheme, $M_\infty = 0.4$

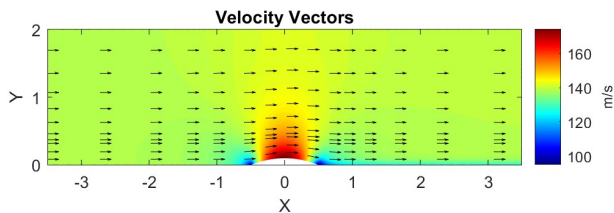


Fig. 33: Velocity Vector Plot, Roe Scheme, $M_\infty = 0.4$

5.2.2 $M_\infty = 0.8$: Due to a large number of oscillations caused by high CFL in the previous, the CFL number was reduced to increase stability of the solver.

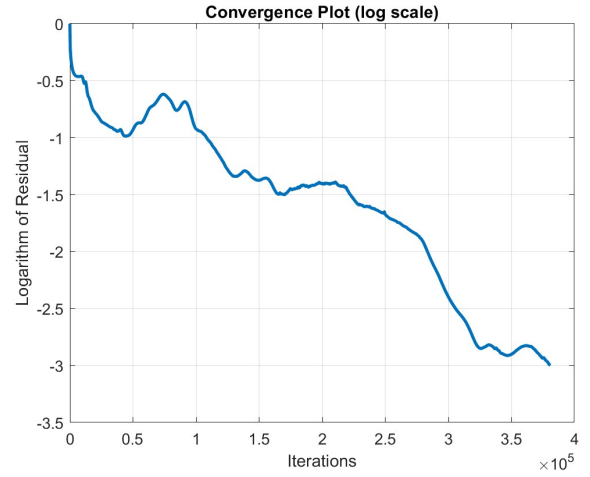


Fig. 34: Convergence Plot, Roe Scheme, $M_\infty = 0.8$

Convergence took more iterations however didn't show much oscillations.

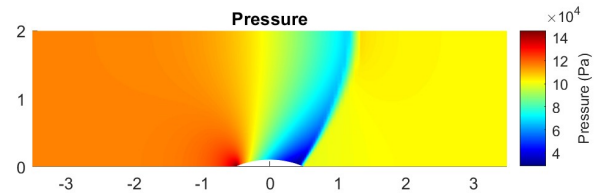


Fig. 35: Pressure Contour, Roe Scheme, $M_\infty = 0.8$

The pressure contour mostly matches with the AUSM scheme. We can notice oscillations (smeared) near the discontinuity for the Roe scheme. This is likely due to the use of a 2nd-order spatial scheme.

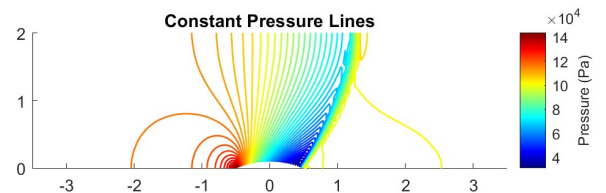


Fig. 36: Constant Pressure Lines, Roe Scheme, $M_\infty = 0.8$

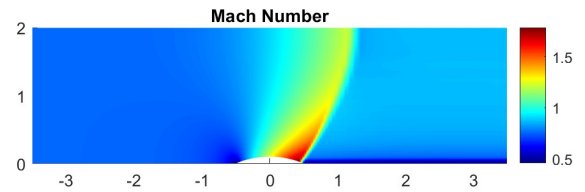


Fig. 37: Mach Number contour, Roe Scheme, $M_\infty = 0.8$

Overall, the Mach number contour looks the same as the AUSM scheme, but again the plots look smeared at the discontinuity.

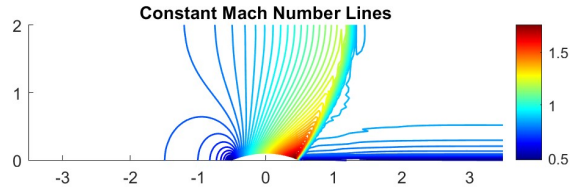


Fig. 38: Mach Number Contour Lines, Roe Scheme, $M_\infty = 0.8$

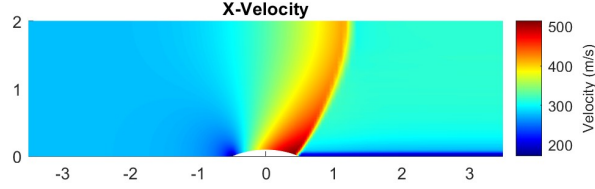


Fig. 39: X-velocity contour, Roe Scheme, $M_\infty = 0.8$

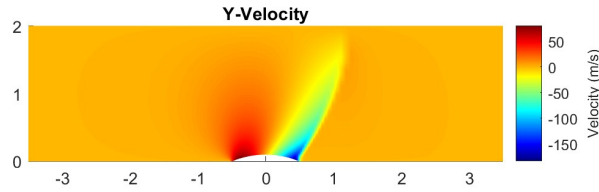


Fig. 40: Y-velocity contour, Roe Scheme, $M_\infty = 0.8$

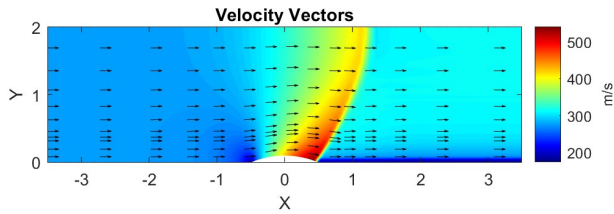


Fig. 41: Velocity Vector Plot, Roe Scheme, $M_\infty = 0.8$

5.2.3 $M_\infty = 1.4$: The CFL was set the same as the previous case of $M_\infty = 0.8$

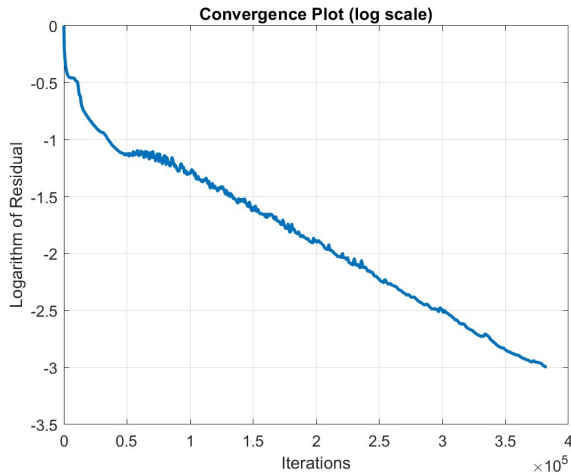


Fig. 42: Convergence Plot, Roe Scheme, $M_\infty = 1.4$

The convergence plot showed more oscillations, but took almost similar number of iterations to converge. The results look quite similar to those obtained using AUSM scheme and first order accuracy, however some smearing is seen near the discontinuities similar to those for $M_\infty = 0.8$.

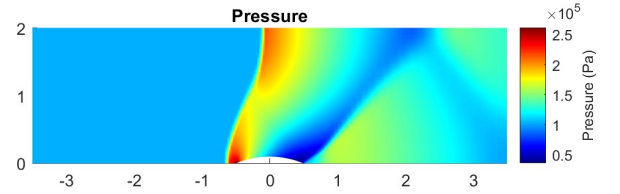


Fig. 43: Pressure Contour, Roe Scheme, $M_\infty = 1.4$

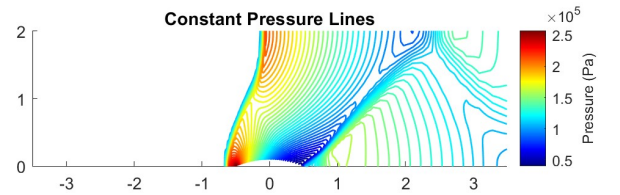


Fig. 44: Constant Pressure Lines, Roe Scheme, $M_\infty = 1.4$

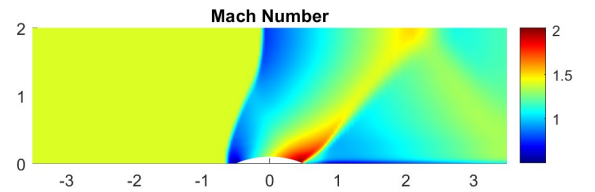


Fig. 45: Mach Number contour, Roe Scheme, $M_\infty = 1.4$

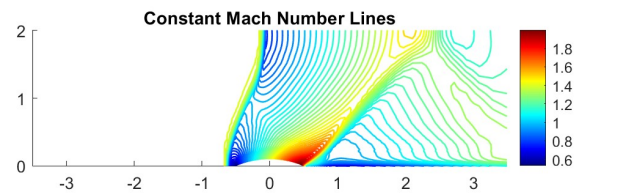


Fig. 46: Mach Number Contour Lines, Roe Scheme, $M_\infty = 1.4$

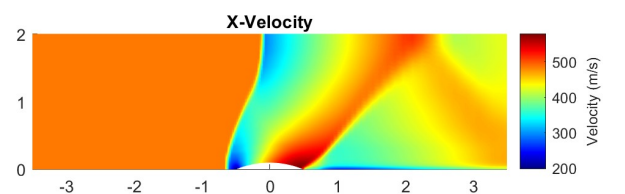


Fig. 47: X-velocity contour, Roe Scheme, $M_\infty = 1.4$

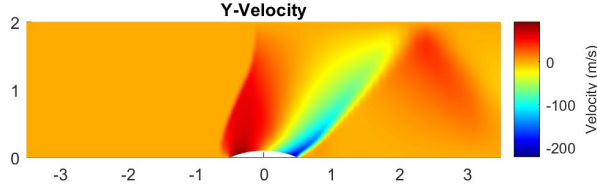


Fig. 48: Y-velocity contour, Roe Scheme, $M_\infty = 1.4$

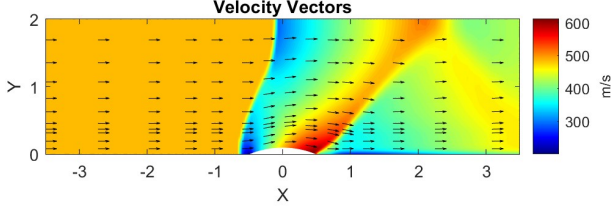


Fig. 49: Velocity Vector Plot, Roe Scheme, $M_\infty = 1.4$

6 Conclusion

A compressible Euler solver was implemented using AUSM, Van Leer and Roe schemes to determine the interface fluxes. Flow over a bump geometry was simulated for freestream Mach numbers 0.4, 0.8 and 1.4 with $p_\infty = 0.01$ atmospheres. The first-order time-marching scheme was extended to second-order temporal accuracy using Heun's scheme, and higher-order spatial reconstruction was performed using limiters such as minmod, Superbee and Van Leer. Results from the solver employing first-order time and space reconstruction with the AUSM scheme, as well as second-order time and space reconstruction with the Roe scheme, were presented.

The convergence plots for $M_\infty = 0.4$ showed more oscillations; this could be because such flux formulations are primarily developed for higher Mach number regimes. The results obtained for this case were very similar to those from an incompressible Euler solver for the same geometry. High pressures were observed at the leading and trailing edges of the bump, corresponding to the expected stagnation regions, and thus low velocities were obtained in these areas. As expected, low pressure and high tangential velocity were observed near the top of the bump.

For the case with $M_\infty = 0.8$, a shock was observed at the end of the bump where the Mach number became supersonic. The shock was strong enough to bring the flow back to subsonic conditions on the downstream side. For $M_\infty = 1.4$, a shock was observed to form upstream of the bump; the drop in pressure and Mach number across this shock approximately matched theoretical predictions for a strong shock. The flow became supersonic again over the bump, leading to another shock similar to that in the $M_\infty = 0.8$ case.

The two different flux schemes produced largely similar results. Using higher-order spatial and temporal reconstruction required lower CFL numbers to prevent divergence; it also led to gradients in the plots being smeared out, since limiters such as minmod introduce significant numerical dissipation that smooth sharp flow features.

Appendix

Code

```
1
2 clc; clear; close all;
3 %% Flow parameters
4 gamma = 1.4;
5 R = 287; %J/kg.K
6 p_atm = 1.01325*1e5;
7
8 M_inf = 0.8;
9 v_inf = 0;
10 p_inf = 1.01*1.01325*1e5;
11 T_inf = 300;
12 u_inf = M_inf*sqrt(gamma*R*T_inf);
13 rho_inf = (p_inf)/(R*T_inf);
14
15 %% Modelling parameters
16 CFL = 0.2;
17 tol = 1e-3;
18 max_iter = 1e10;
19
20 % Reset flag so that if program terminates before converging then we dont
21 % have to run from start again
22 % 1 : Start from 0th iteration and want the node values to be backed up every 100 iteration
23 % 0 : Start from previous terminated point and uses u, v, p from backup_file.txt
24 % -1 : Starts from 0th iteration and no backup created
25 reset = -1;
26
27 % For switching between order of reconstruction
28 func = 0;
29 % 0 - First order
30 % 1 - Higher order - superbee
31 % 2 - Higher order - minmod
32 % 3 - Higher order - van leer
33
34 % For switching between interface flux schemes
35 scheme = 2;
36 % 0 - Roe Scheme
37 % 1 - Van Leer Scheme
38 % 2 - AUSM Scheme
39
40 % For higher order time resolution
41 time_resolution = 0;
42 % 0 - first order
43 % 1 - RK2 Heun's Scheme
44
45 %% Reading and storing mesh data
46 data = readtable('bumpgrid.dat.txt');
47 x = data(:,1); y = data(:,2);
48 imax = int32(x(1)); jmax = int32(y(1));
49 x = x(2:end); y = y(2:end);
50 mesh_data_x = zeros(imax,jmax); mesh_data_y = zeros(imax,jmax);
51 line = 1;
52 for j = 1:jmax
53     for i = 1:imax
54         mesh_data_x(i,j) = x(line);
55         mesh_data_y(i,j) = y(line);
56         line = line + 1;
57     end
58 end
59 %% Mesh Visualization
60 fprintf('-----Plotting Mesh-----\n\n')
61
62 figure;
63 hold on;
64
65 plot(mesh_data_x, mesh_data_y, '-g')
66 plot(mesh_data_x', mesh_data_y', '-g')
67 title('Mesh Domain'); xlabel('X'); ylabel('Y');
68 axis equal tight
69
```

```

70 fprintf('-----Mesh Plot Done-----\n\n')
71
72 %% Finding normal vector for faces
73 n_xface_x = zeros(imax,jmax - 1); n_xface_y = zeros(imax,jmax - 1);
74 n_yface_x = zeros(imax-1,jmax); n_yface_y = zeros(imax-1,jmax);
75 A_x = zeros(imax,jmax-1); A_y = zeros(imax-1,jmax);
76
77 % y face
78 % (i,j) will be area or normal of y-face bw pts (i,j) and (i+1,j)
79 for j = 1 : jmax
80     for i = 1 : imax - 1
81         t_x = mesh_data_x(i,j) - mesh_data_x(i+1,j);
82         t_y = mesh_data_y(i,j) - mesh_data_y(i+1,j);
83         n_yface_x(i,j) = t_y; n_yface_y(i,j) = -t_x;
84         A_y(i,j) = sqrt(t_x^2 + t_y^2);
85     end
86 end
87 % normalisation
88 n_yface_x = n_yface_x./A_y; n_yface_y = n_yface_y./A_y;
89 % x face
90 % (i,j) will be area or normal of x-face bw pts (i,j) and (i,j+1)
91 for j = 1 : jmax - 1
92     for i = 1 : imax
93         t_x = mesh_data_x(i,j+1) - mesh_data_x(i,j);
94         t_y = mesh_data_y(i,j+1) - mesh_data_y(i,j);
95         n_xface_x(i,j) = t_y; n_xface_y(i,j) = -t_x;
96         A_x(i,j) = sqrt(t_x^2 + t_y^2);
97     end
98 end
99 % normalisation
100 n_xface_x = n_xface_x./A_x; n_xface_y = n_xface_y./A_x;
101
102 %% Variable Initialisation
103 F1 = zeros(imax,jmax-1);
104 F2 = zeros(imax,jmax-1);
105 F3 = zeros(imax,jmax-1);
106 F4 = zeros(imax,jmax-1);
107 G1 = zeros(imax-1,jmax);
108 G2 = zeros(imax-1,jmax);
109 G3 = zeros(imax-1,jmax);
110 G4 = zeros(imax-1,jmax);
111 if (reset == 1)
112     u = u_inf*ones(imax+1, jmax+1);
113     v = v_inf*ones(imax+1, jmax+1);
114     p = p_inf*ones(imax+1, jmax+1);
115     rho = rho_inf*ones(imax+1, jmax+1);
116     fid_resnorm = fopen("backup_resnorm.txt",'w');
117     fclose(fid_resnorm);
118 elseif (reset == -1)
119     u = u_inf*ones(imax+1, jmax+1);
120     v = v_inf*ones(imax+1, jmax+1);
121     p = p_inf*ones(imax+1, jmax+1);
122     rho = rho_inf*ones(imax+1, jmax+1);
123 elseif (reset == 0)
124     fprintf('-----Reading from backup files-----\n\n')
125     prev_data = readtable('backup_file.txt');
126     prev_rho = prev_data(:,1); prev_u = prev_data(:,2); prev_v = prev_data(:,3); prev_p =
        ↪ prev_data(:,4);
127     u = zeros(imax+1, jmax+1);
128     v = zeros(imax+1, jmax+1);
129     p = zeros(imax+1, jmax+1);
130     rho = zeros(imax+1, jmax+1);
131     line = 1;
132     for j = 1:jmax+1
133         for i = 1:imax+1
134             u(i,j) = prev_u(line);
135             v(i,j) = prev_v(line);
136             p(i,j) = prev_p(line);
137             rho(i,j) = prev_rho(line);
138             line = line + 1;
139         end
140     end
141     prev_data = readtable('backup_resnorm.txt');

```

```

142     prev_itn = prev_data(:,1); prev_resnorm = prev_data(:,2);
143 end
144
145
146 V_1 = rho; V_2 = u; V_3 = v; V_4 = p;
147 T = V_4./(R*V_1);
148
149 % Mach Number
150 M_x = u./sqrt(gamma*R*T); M_y = v./sqrt(gamma*R*T);
151
152 % This is for higher order reconstructions
153 V1_xface_L = zeros(imax,jmax-1);
154 V2_xface_L = zeros(imax,jmax-1);
155 V3_xface_L = zeros(imax,jmax-1);
156 V4_xface_L = zeros(imax,jmax-1);
157
158 V1_xface_R = zeros(imax,jmax-1);
159 V2_xface_R = zeros(imax,jmax-1);
160 V3_xface_R = zeros(imax,jmax-1);
161 V4_xface_R = zeros(imax,jmax-1);
162
163 V1_yface_U = zeros(imax-1,jmax);
164 V2_yface_U = zeros(imax-1,jmax);
165 V3_yface_U = zeros(imax-1,jmax);
166 V4_yface_U = zeros(imax-1,jmax);
167
168 V1_yface_D = zeros(imax-1,jmax);
169 V2_yface_D = zeros(imax-1,jmax);
170 V3_yface_D = zeros(imax-1,jmax);
171 V4_yface_D = zeros(imax-1,jmax);
172
173
174 %% Main loop
175 fprintf('-----Entering main loop-----\n\n')
176 if ((reset == 1) || (reset == -1))
177     itn = 1;
178 elseif (reset == 0)
179     itn = prev_itn(end);
180 end
181
182 resnorm = [];
183 while true
184
185     if time_resolution == 0
186         [Res1,Res2,Res3,Res4,dt_by_Vol,V_1,V_2,V_3,V_4,F1,F2,F3,F4,G1,G2,G3,G4] = ...
187         computeResidual( v_inf,u_inf,p_inf,rho_inf,...
188             V_1,V_2,V_3,V_4, ...
189             M_x,M_y,...
190             F1,F2,F3,F4,G1,G2,G3,G4,...
191             V1_xface_L, V1_xface_R, V1_yface_U, V1_yface_D,...
192             V2_xface_L, V2_xface_R, V2_yface_U, V2_yface_D,...
193             V3_xface_L, V3_xface_R, V3_yface_U, V3_yface_D,...
194             V4_xface_L, V4_xface_R, V4_yface_U, V4_yface_D,...
195             imax,jmax,CFL, ...
196             n_xface_x,n_xface_y,n_yface_x,n_yface_y, ...
197             A_x,A_y,func,scheme);
198
199         %% Update loop
200         f = 1; %relaxation factor
201         V_1_old = V_1; V_2_old = V_2; V_3_old = V_3; V_4_old = V_4;
202         Q_1 = V_1; Q_2 = V_1.*V_2; Q_3 = V_1.*V_3; Q_4 = V_4/(gamma-1)+V_1/2.*(V_2.*V_2 +
203             ↪ V_3.*V_3);
204
205         Q_1(2:imax,2:jmax) = Q_1(2:imax,2:jmax) - Res1.*dt_by_Vol;
206         Q_2(2:imax,2:jmax) = Q_2(2:imax,2:jmax) - Res2.*dt_by_Vol;
207         Q_3(2:imax,2:jmax) = Q_3(2:imax,2:jmax) - Res3.*dt_by_Vol;
208         Q_4(2:imax,2:jmax) = Q_4(2:imax,2:jmax) - Res4.*dt_by_Vol;
209
210         V_1 = f*Q_1 + (1-f)*V_1_old;
211         V_2 = f*Q_2./Q_1 + (1-f)*V_2_old;
212         V_3 = f*Q_3./Q_1 + (1-f)*V_3_old;
213         V_4 = f*(gamma - 1)*(Q_4 - 0.5*((Q_2.*Q_2+Q_3.*Q_3)./Q_1))+(1-f)*V_4_old;
214         T = (1/R)*V_4./V_1;

```

```

214     M_x = V_2./sqrt(gamma*R*T); M_y = V_3./sqrt(gamma*R*T);
215     M = sqrt(M_x.*M_x + M_y.*M_y);
216
217 elseif time_resolution == 1
218     [Res1,Res2,Res3,Res4,dt_by_Vol1,V_1,V_2,V_3,V_4,F1,F2,F3,F4,G1,G2,G3,G4] = ...
219     computeResidual( v_inf,u_inf,p_inf,rho_inf,...
220                     V_1,V_2,V_3,V_4, ...
221                     M_x,M_y,...
222                     F1,F2,F3,F4,G1,G2,G3,G4,...
223                     V1_xface_L, V1_xface_R, V1_yface_U, V1_yface_D,...
224                     V2_xface_L, V2_xface_R, V2_yface_U, V2_yface_D,...
225                     V3_xface_L, V3_xface_R, V3_yface_U, V3_yface_D,...
226                     V4_xface_L, V4_xface_R, V4_yface_U, V4_yface_D,...
227                     imax,jmax,CFL, ...
228                     n_xface_x,n_xface_y,n_yface_x,n_yface_y, ...
229                     A_x,A_y,func,scheme);
230
231     % conservative variables from primitives (using BC'd state V_*b)
232     Q1 = V_1;
233     Q2 = V_1 .* V_2;
234     Q3 = V_1 .* V_3;
235     Q4 = V_4/(gamma-1) + 0.5*V_1 .* (V_2.^2 + V_3.^2);
236
237     % k1 = -Res * dt1 (only on interior cells)
238     k1_1 = zeros(size(Q1));
239     k1_2 = zeros(size(Q2));
240     k1_3 = zeros(size(Q3));
241     k1_4 = zeros(size(Q4));
242
243     k1_1(2:imax,2:jmax) = -Res1 .* dt_by_Vol1;
244     k1_2(2:imax,2:jmax) = -Res2 .* dt_by_Vol1;
245     k1_3(2:imax,2:jmax) = -Res3 .* dt_by_Vol1;
246     k1_4(2:imax,2:jmax) = -Res4 .* dt_by_Vol1;
247
248     % intermediate state Q* = Q + k1
249     Q1s = Q1; Q2s = Q2; Q3s = Q3; Q4s = Q4;
250     Q1s(2:imax,2:jmax) = Q1(2:imax,2:jmax) + k1_1(2:imax,2:jmax);
251     Q2s(2:imax,2:jmax) = Q2(2:imax,2:jmax) + k1_2(2:imax,2:jmax);
252     Q3s(2:imax,2:jmax) = Q3(2:imax,2:jmax) + k1_3(2:imax,2:jmax);
253     Q4s(2:imax,2:jmax) = Q4(2:imax,2:jmax) + k1_4(2:imax,2:jmax);
254
255     V_1s = Q1s;
256     V_2s = Q2s ./ Q1s;
257     V_3s = Q3s ./ Q1s;
258     V_4s = (gamma - 1) .* (Q4s - 0.5*((Q2s.^2 + Q3s.^2)./Q1s));
259
260     [Res1s,Res2s,Res3s,Res4s,dt_by_Vol2,V_1,V_2,V_3,V_4,F1,F2,F3,F4,G1,G2,G3,G4] = ...
261     computeResidual( v_inf,u_inf,p_inf,rho_inf,...
262                     V_1s,V_2s,V_3s,V_4s, ...
263                     M_x,M_y,...
264                     F1,F2,F3,F4,G1,G2,G3,G4,...
265                     V1_xface_L, V1_xface_R, V1_yface_U, V1_yface_D,...
266                     V2_xface_L, V2_xface_R, V2_yface_U, V2_yface_D,...
267                     V3_xface_L, V3_xface_R, V3_yface_U, V3_yface_D,...
268                     V4_xface_L, V4_xface_R, V4_yface_U, V4_yface_D,...
269                     imax,jmax,CFL, ...
270                     n_xface_x,n_xface_y,n_yface_x,n_yface_y, ...
271                     A_x,A_y,func,scheme);
272
273     % k2 = -Res* dt2
274     k2_1 = zeros(size(Q1));
275     k2_2 = zeros(size(Q2));
276     k2_3 = zeros(size(Q3));
277     k2_4 = zeros(size(Q4));
278
279     k2_1(2:imax,2:jmax) = -Res1s .* dt_by_Vol2;
280     k2_2(2:imax,2:jmax) = -Res2s .* dt_by_Vol2;
281     k2_3(2:imax,2:jmax) = -Res3s .* dt_by_Vol2;
282     k2_4(2:imax,2:jmax) = -Res4s .* dt_by_Vol2;
283
284     % ===== Heun update: Q^{n+1} = Q^n + 0.5*k1 + 0.5*k2 =====
285     Q1(2:imax,2:jmax) = Q1(2:imax,2:jmax) + 0.5*(k1_1(2:imax,2:jmax) +
    ↪ k2_1(2:imax,2:jmax));

```

```

286     Q2(2:imax,2:jmax) = Q2(2:imax,2:jmax) + 0.5*(k1_2(2:imax,2:jmax) +
287         ↪ k2_2(2:imax,2:jmax));
288     Q3(2:imax,2:jmax) = Q3(2:imax,2:jmax) + 0.5*(k1_3(2:imax,2:jmax) +
289         ↪ k2_3(2:imax,2:jmax));
290     Q4(2:imax,2:jmax) = Q4(2:imax,2:jmax) + 0.5*(k1_4(2:imax,2:jmax) +
291         ↪ k2_4(2:imax,2:jmax));
292
293     % back to primitives for new time level
294     V_1 = Q1;
295     V_2 = Q2 ./ Q1;
296     V_3 = Q3 ./ Q1;
297     V_4 = (gamma - 1) .* (Q4 - 0.5*((Q2.^2 + Q3.^2)./Q1));
298
299     % for convergence check, you can reuse the last residual (Res1s..)
300     % and dt2 (call it dt_by_Vol = dt2) in your residual norm block
301     Res1 = Res1s;
302     Res2 = Res2s;
303     Res3 = Res3s;
304     Res4 = Res4s;
305
306 end
307
308 %% Residual Norm calculations
309
310 resnorm(itn) = 0;
311
312 scale1 = rho_inf*u_inf;
313 scale2 = rho_inf*u_inf*u_inf;
314 scale3 = rho_inf*u_inf*u_inf;
315 h0_inf = gamma/(gamma-1)*(p_inf/rho_inf) + u_inf^2/2;
316 scale4 = rho_inf*u_inf*h0_inf;
317
318 for j = 2:jmax
319     for i = 2:imax
320         resnorm(itn) = resnorm(itn) + (Res1(i-1,j-1)/scale1)^2 + (Res2(i-1,j-1)/scale2)^2
321         ↪ ...
322         + (Res3(i-1,j-1)/scale3)^2 + (Res4(i-1,j-1)/scale4)^2;
323     end
324 end
325
326 resnorm(itn) = sqrt(resnorm(itn));
327
328 if (reset == -1)
329     if itn == 1
330         resnorm_0 = resnorm(itn);
331     end
332 end
333
334 if (reset == 1)
335     if itn == 1
336         resnorm_0 = resnorm(itn);
337         fid_resnorm = fopen('backup_resnorm.txt','a');
338         fprintf(fid_resnorm, '%f %f\n', 0, resnorm_0);
339         fclose(fid_resnorm);
340     end
341 elseif (reset == 0)
342     if itn == prev_itn(end)
343         prev_data = readtable('backup_resnorm.txt');
344         resnorm_0 = prev_data{1,2};
345     end
346 end
347
348 if (resnorm(itn)/resnorm_0) < tol
349     break
350 end
351
352 if (mod(itn,10) == 0)
353     fprintf('Iteration : %d | Resnorm Ratio : %f\n', itn, resnorm(itn)/resnorm_0)
354 end
355
356 if ((mod(itn,100) == 0) && (reset ~= -1))
357     % Every 100 iterations, x velocity, y velocity and pressure will be
358     % stored in 'backup_file.txt'. In case of program terminating before

```

```

355     % convergence we can use this backup file to start off from n*100 iteration
356     % by setting reset variable as 0, rather from scratch again.
357     fid_backup = fopen("backup_file.txt",'w');
358     fprintf('-----Writing the backup file-----\n\n')
359     for j = 1:jmax+1
360         for i = 1:imax+1
361             fprintf(fid_backup,'%f %f %f %f\n', V_1(i,j), V_2(i,j), V_3(i,j), V_4(i,j));
362         end
363     end
364     fclose(fid_backup);
365     fid_resnorm = fopen('backup_resnorm.txt','a');
366     fprintf(fid_resnorm, '%d %f\n', itn, resnorm(itn));
367     fclose(fid_resnorm);
368 end
369 if itn > max_iter && max_iter ~= -1
370     fprintf('----- Maximum Iterations Reached
371     ↪ -----\n\n')
372     break;
373 end
374 itn = itn + 1;
375 end
376
377 %% Convergence Plot
378 figure;
379 plot((1:itn), log10(resnorm/resnorm_0), '- ',LineWidth=2);
380 xlabel('Iterations');
381 ylabel('Logarithm of Residual');
382 title('Convergence Plot (log scale)')
383 grid on;
384
385 %% Reimposing Boundary Conditions
386 % Inflow
387 for j = 2 : jmax
388     if M_x(1,j) >=1
389         V_1(1,j) =rho_inf;
390         V_2(1,j) = u_inf;
391         V_3(1,j) = v_inf;
392         V_4(1,j) = p_inf;
393     elseif M_x(1,j) < 1
394         V_1(1,j) =rho_inf;
395         V_2(1,j) = u_inf;
396         V_3(1,j) = v_inf;
397         V_4(1,j) = V_4(2,j);
398     end
399 end
400
401 % Outflow
402 for j = 2 : jmax
403     if M_x(imax+1,j) >=1
404         V_1(imax+1,j) = V_1(imax,j);
405         V_2(imax+1,j) = V_2(imax,j);
406         V_3(imax+1,j) = V_3(imax,j);
407         V_4(imax+1,j) = V_4(imax,j);
408     elseif M_x(imax+1,j) < 1
409         V_1(imax+1,j) =V_1(imax,j);
410         V_2(imax+1,j) = V_2(imax,j);
411         V_3(imax+1,j) = V_3(imax,j);
412         V_4(imax+1,j) = p_inf;
413     end
414 end
415
416 % Bottom Wall
417 G1(:,1) = 0;
418 V_1(:,1) = V_1(:,2);
419 V_4(:,1) = V_4(:,2);
420 U_contrav_lower = V_2(2:imax,2).*n_yface_x(:,1) + V_3(2:imax,2).*n_yface_y(:,1);
421 V_2(2:imax,1) = V_2(2:imax, 2) - 2*U_contrav_lower.*n_yface_x(:,1);
422 V_3(2:imax,1) = V_3(2:imax, 2) - 2*U_contrav_lower.*n_yface_y(:,1);
423
424 % Top Wall
425 G1(:,jmax) = 0;
426 V_1(:,jmax+1) = V_1(:,jmax);

```

```

427 V_4(:, jmax+1) = V_4(:, jmax);
428 U_contrav_upper = V_2(2:imax, jmax).*n_yface_x(:, jmax) + V_3(2:imax, jmax).*n_yface_y(:, jmax);
429 V_2(2:imax, jmax+1) = V_2(2:imax, jmax) - 2*U_contrav_upper.*n_yface_x(:, jmax);
430 V_3(2:imax, jmax+1) = V_3(2:imax, jmax) - 2*U_contrav_upper.*n_yface_y(:, jmax);
431
432 % Bottom-left corner
433 V_1(1,1) = V_1(2,2);
434 V_2(1,1) = V_2(2,2);
435 V_3(1,1) = V_3(2,2);
436 V_4(1,1) = V_4(2,2);
437
438 % Bottom-right corner
439 V_1(imax+1,1) = V_1(imax,2);
440 V_2(imax+1,1) = V_2(imax,2);
441 V_3(imax+1,1) = V_3(imax,2);
442 V_4(imax+1,1) = V_4(imax,2);
443
444 % Top-left corner
445 V_1(1, jmax+1) = V_1(2, jmax);
446 V_2(1, jmax+1) = V_2(2, jmax);
447 V_3(1, jmax+1) = V_3(2, jmax);
448 V_4(1, jmax+1) = V_4(2, jmax);
449
450 % Top-right corner
451 V_1(imax+1, jmax+1) = V_1(imax, jmax);
452 V_2(imax+1, jmax+1) = V_2(imax, jmax);
453 V_3(imax+1, jmax+1) = V_3(imax, jmax);
454 V_4(imax+1, jmax+1) = V_4(imax, jmax);
455 %% Post - Processing
456 fprintf('-----Solution Converged-----\n\n')
457 fprintf('-----Post Processing-----\n\n')
458 str1 = datetime('now', 'Format', 'yyyy_MM_dd_HH_mm_ss');
459 str2 = sprintf('%f', CFL);
460 % File name format 'year_month_day_hour_min_sec_alpha_beta_CFL.txt'
461 output_file_name = string(str1) + str2 + '.txt';
462 fid_output = fopen(output_file_name, 'w');
463 for j = 1:jmax
464     for i = 1:imax
465         u_node = 0.25*(V_2(i, j) + V_2(i+1, j) + V_2(i+1, j+1) + V_2(i, j+1));
466         v_node = 0.25*(V_3(i, j) + V_3(i+1, j) + V_3(i+1, j+1) + V_3(i, j+1));
467         p_node = 0.25*(V_4(i, j) + V_4(i+1, j) + V_4(i+1, j+1) + V_4(i, j+1));
468         rho_node = 0.25*(V_1(i, j) + V_1(i+1, j) + V_1(i+1, j+1) + V_1(i, j+1));
469         fprintf(fid_output, '%f %f %f %f\n', rho_node, u_node, v_node, p_node);
470     end
471 end
472
473 fclose(fid_output);
474 u_node = zeros(imax, jmax);
475 v_node = zeros(imax, jmax);
476 p_node = zeros(imax, jmax);
477 rho_node = zeros(imax, jmax);
478 output_data = readtable(output_file_name);
479 output_rho = output_data(:,1); output_u = output_data(:,2);
480 output_v = output_data(:,3); output_p = output_data(:,4);
481
482 line = 1;
483 for j = 1:jmax
484     for i = 1:imax
485         u_node(i, j) = output_u(line);
486         v_node(i, j) = output_v(line);
487         p_node(i, j) = output_p(line);
488         rho_node(i, j) = output_rho(line);
489         line = line + 1;
490     end
491 end
492
493 T_node = p_node./(rho_node*R);
494 M_node = sqrt(u_node.*u_node + v_node.*v_node)./sqrt(gamma*R*T_node);
495 fprintf('-----Plotting Results-----\n\n')
496 figure; hold on
497 surf(mesh_data_x, mesh_data_y, u_node);
498 h = colorbar;
499 ylabel(h, 'Velocity (m/s)');

```

```

500 colormap('jet')
501 shading interp
502 title('X-Velocity')
503 axis equal
504 axis tight
505
506 figure; hold on
507 surf(mesh_data_x, mesh_data_y, v_node);
508 h = colorbar;
509 ylabel(h, 'Velocity (m/s)');
510 colormap('jet')
511 shading interp
512 title('Y-Velocity')
513 axis equal
514 axis tight
515
516 figure; hold on
517 surf(mesh_data_x, mesh_data_y, p_node);
518 h = colorbar;
519 ylabel(h, 'Pressure (Pa)');
520 colormap('jet')
521 shading interp
522 title('Pressure')
523 axis equal
524 axis tight
525
526 figure; hold on
527 contour(mesh_data_x, mesh_data_y, p_node, 50, 'LineWidth', 1);
528 h = colorbar;
529 ylabel(h, 'Pressure (Pa)');
530 colormap('jet')
531 shading interp
532 title('Constant Pressure Lines')
533 axis equal
534 axis tight
535
536 figure; hold on
537 surf(mesh_data_x, mesh_data_y, rho_node);
538 h = colorbar;
539 ylabel(h, 'Density (kg/m^3)');
540 colormap('jet')
541 shading interp
542 title('Density')
543 axis equal
544 axis tight
545
546 figure; hold on
547 surf(mesh_data_x, mesh_data_y, T_node);
548 h = colorbar;
549 ylabel(h, 'Temperature (K)');
550 colormap('jet')
551 shading interp
552 title('Temperature')
553 axis equal
554 axis tight
555
556 figure; hold on
557 surf(mesh_data_x, mesh_data_y, M_node);
558 colorbar
559 colormap('jet')
560 shading interp
561 title('Mach Number')
562 axis equal
563 axis tight
564
565 figure; hold on
566 contour(mesh_data_x, mesh_data_y, M_node, 50, 'LineWidth', 1);
567 colorbar
568 colormap('jet')
569 shading interp
570 title('Constant Mach Number Lines')
571 axis equal
572 axis tight

```

```

573
574 figure;
575 ylines = [5, 10, 17, 15, 20, 25, 30, 35, 40, 45]; xlines = [1, 5, 10, 15, 20, 30, 40, 50, 60,
↵ 70, 75, 80, 85 90, 95];
576 contourf(mesh_data_x, mesh_data_y, sqrt(u_node.^2 + v_node.^2), 100, 'LineStyle', 'none'); hold
↵ on
577 quiver(mesh_data_x(xlines, ylines), mesh_data_y(xlines, ylines), u_node(xlines,
↵ ylines)./sqrt(u_node(xlines, ylines).^2 + v_node(xlines, ylines).^2), v_node(xlines,
↵ ylines)./sqrt(u_node(xlines, ylines).^2 + v_node(xlines, ylines).^2), 0.25,
↵ 'k', 'ShowArrowHead', 'on')
578 a = colorbar;
579 ylabel(a, 'm/s')
580 colormap('jet')
581 shading interp
582 title('Velocity Vectors'); xlabel('X'); ylabel('Y')
583 axis equal
584 axis tight
585
586
587
588 %% Functions
589 function output = sgn(M)
590     if M >= 0
591         output = 1;
592     elseif M < 0
593         output = -1;
594     end
595 end
596
597 function output = limiter(a,b,func)
598
599     switch func
600     case 0
601         output = 0;
602     case 1
603         output = 0.5*(sign(a)+sign(b))*min([max(abs(a),abs(b)),2*abs(a),2*abs(b)]);
604         % superbee
605     case 2
606         if a*b <= 0
607             output = 0;
608         elseif abs(a)<=abs(b)
609             output = a;
610         elseif abs(b) < abs(a)
611             output = b;
612         end
613         % minmod
614     case 3
615         epsilon = 1e-16;
616         output = ((a*abs(b)+b*abs(a))/(abs(a)+abs(b)+epsilon));
617         % van leer
618     end
619
620 end
621
622 function [F1,F2,F3,F4,G1,G2,G3,G4] = Roe_Scheme( ...
623     F1i,F2i,F3i,F4i,G1i,G2i,G3i,G4i,A_x,A_y, ...
624     imax, jmax, n_xface_x, n_xface_y, n_yface_x, n_yface_y, ...
625     V1_xface_L, V1_xface_R, V2_xface_L, V2_xface_R, ...
626     V3_xface_L, V3_xface_R, V4_xface_L, V4_xface_R, ...
627     V1_yface_D, V1_yface_U, V2_yface_D, V2_yface_U, ...
628     V3_yface_D, V3_yface_U, V4_yface_D, V4_yface_U)
629
630     F1 = F1i;
631     F2 = F2i;
632     F3 = F3i;
633     F4 = F4i;
634     G1 = G1i;
635     G2 = G2i;
636     G3 = G3i;
637     G4 = G4i;
638
639     gamma = 1.4;
640     % For x - face

```

```

641 for j = 1 : jmax - 1
642     for i = 1 : imax
643         rho_l = V1_xface_L(i, j);
644         rho_r = V1_xface_R(i, j);
645         del_rho = rho_r - rho_l;
646
647         u_l = V2_xface_L(i, j);
648         u_r = V2_xface_R(i, j);
649         del_u = u_r - u_l;
650
651         v_l = V3_xface_L(i, j);
652         v_r = V3_xface_R(i, j);
653         del_v = v_r - v_l;
654
655         p_l = V4_xface_L(i, j);
656         p_r = V4_xface_R(i, j);
657         del_p = p_r - p_l;
658
659         h0_l = gamma/(gamma-1)*(p_l/rho_l) + 0.5*(u_l^2 + v_l^2);
660         h0_r = gamma/(gamma-1)*(p_r/rho_r) + 0.5*(u_r^2 + v_r^2);
661
662         unl = u_l*n_xface_x(i, j) + v_l*n_xface_y(i, j);
663         unr = u_r*n_xface_x(i, j) + v_r*n_xface_y(i, j);
664         del_un = unr - unl;
665
666         r = sqrt(rho_r/rho_l);
667
668         rho_R = sqrt(rho_r*rho_l);
669
670         u_R = (u_l + u_r*r) / (1 + r);
671         v_R = (v_l + v_r*r) / (1 + r);
672
673         h0_R = (h0_l + h0_r*r) / (1 + r);
674
675         a_R = sqrt((gamma - 1)*(h0_R - 0.5*(u_R^2 + v_R^2)));
676
677         U_contrav_R = u_R*n_xface_x(i, j) + v_R*n_xface_y(i, j);
678
679         alpha_1 = A_x(i, j)*abs(U_contrav_R)*(del_rho - del_p/a_R^2);
680         alpha_2 = A_x(i, j)/(2*a_R^2) * abs(U_contrav_R + a_R) * (del_p +
        ↪ rho_R*a_R*del_un);
681         alpha_3 = A_x(i, j)/(2*a_R^2) * abs(U_contrav_R - a_R) * (del_p -
        ↪ rho_R*a_R*del_un);
682         alpha_4 = alpha_1 + alpha_2 + alpha_3;
683         alpha_5 = a_R*(alpha_2 - alpha_3);
684         alpha_6 = A_x(i, j)*abs(U_contrav_R)*(rho_R*del_u - n_xface_x(i, j)*rho_R*del_un);
685         alpha_7 = A_x(i, j)*abs(U_contrav_R)*(rho_R*del_v - n_xface_y(i, j)*rho_R*del_un);
686
687         d_1 = alpha_4;
688         d_2 = u_R*alpha_4 + n_xface_x(i, j)*alpha_5 + alpha_6;
689         d_3 = v_R*alpha_4 + n_xface_y(i, j)*alpha_5 + alpha_7;
690         d_4 = h0_R*alpha_4 + U_contrav_R*alpha_5 + u_R*alpha_6 + v_R*alpha_7 - a_R^2 *
        ↪ alpha_1/(gamma-1);
691
692         fm_p = 0.5*A_x(i, j)*rho_l*unl;
693         fm_m = 0.5*A_x(i, j)*rho_r*unr;
694         F1(i, j) = (fm_p + fm_m) - 0.5*d_1;
695         F2(i, j) = (fm_p*u_l + fm_m*u_r) + 0.5*(p_l + p_r)*A_x(i, j)*n_xface_x(i, j) -
        ↪ 0.5*d_2;
696         F3(i, j) = (fm_p*v_l + fm_m*v_r) + 0.5*(p_l + p_r)*A_x(i, j)*n_xface_y(i, j) -
        ↪ 0.5*d_3;
697         F4(i, j) = (fm_p*h0_l + fm_m*h0_r) - 0.5*d_4;
698     end
699 end
700
701 % For y - face
702 for i = 1 : imax - 1
703     for j = 1 : jmax
704         rho_d = V1_yface_D(i, j);
705         rho_u = V1_yface_U(i, j);
706         del_rho = rho_u - rho_d;
707
708         u_d = V2_yface_D(i, j);

```

```

709     u_u = V2_yface_U(i,j);
710     del_u = u_u - u_d;
711
712     v_d = V3_yface_D(i,j);
713     v_u = V3_yface_U(i,j);
714     del_v = v_u - v_d;
715
716     p_d = V4_yface_D(i,j);
717     p_u = V4_yface_U(i,j);
718     del_p = p_u - p_d;
719
720     h0_d = gamma/(gamma-1)*(p_d/rho_d) + 0.5*(u_d^2 + v_d^2);
721     h0_u = gamma/(gamma-1)*(p_u/rho_u) + 0.5*(u_u^2 + v_u^2);
722
723     und = u_d*n_yface_x(i,j) + v_d*n_yface_y(i,j);
724     unu = u_u*n_yface_x(i,j) + v_u*n_yface_y(i,j);
725     del_un = unu - und;
726
727     r = sqrt(rho_u/rho_d);
728
729     rho_R = sqrt(rho_u*rho_d);
730
731     u_R = (u_d + u_u*r) / (1 + r);
732     v_R = (v_d + v_u*r) / (1 + r);
733
734     h0_R = (h0_d + h0_u*r) / (1 + r);
735
736     a_R = sqrt((gamma - 1)*(h0_R - 0.5*(u_R^2 + v_R^2)));
737
738     U_contrav_R = u_R*n_yface_x(i,j) + v_R*n_yface_y(i,j);
739
740     alpha_1 = A_y(i,j)*abs(U_contrav_R)*(del_rho - del_p/a_R^2);
741     alpha_2 = A_y(i,j)/(2*a_R^2) * abs(U_contrav_R + a_R) * (del_p +
742     ↪ rho_R*a_R*del_un);
743     alpha_3 = A_y(i,j)/(2*a_R^2) * abs(U_contrav_R - a_R) * (del_p -
744     ↪ rho_R*a_R*del_un);
745     alpha_4 = alpha_1 + alpha_2 + alpha_3;
746     alpha_5 = a_R*(alpha_2 - alpha_3);
747     alpha_6 = A_y(i,j)*abs(U_contrav_R)*(rho_R*del_u - n_yface_x(i,j)*rho_R*del_un);
748     alpha_7 = A_y(i,j)*abs(U_contrav_R)*(rho_R*del_v - n_yface_y(i,j)*rho_R*del_un);
749
750     d_1 = alpha_4;
751     d_2 = u_R*alpha_4 + n_yface_x(i,j)*alpha_5 + alpha_6;
752     d_3 = v_R*alpha_4 + n_yface_y(i,j)*alpha_5 + alpha_7;
753     d_4 = h0_R*alpha_4 + U_contrav_R*alpha_5 + u_R*alpha_6 + v_R*alpha_7 - a_R^2 *
754     ↪ alpha_1/(gamma-1);
755
756     fm_p = 0.5*A_y(i,j)*rho_d*und;
757     fm_m = 0.5*A_y(i,j)*rho_u*unu;
758     G1(i,j) = (fm_p + fm_m) - 0.5*d_1;
759     G2(i,j) = (fm_p*u_d + fm_m*u_u) + 0.5*(p_d + p_u)*A_y(i,j)*n_yface_x(i,j) -
760     ↪ 0.5*d_2;
761     G3(i,j) = (fm_p*v_d + fm_m*v_u) + 0.5*(p_d + p_u)*A_y(i,j)*n_yface_y(i,j) -
762     ↪ 0.5*d_3;
763     G4(i,j) = (fm_p*h0_d + fm_m*h0_u) - 0.5*d_4;
764
765     end
766 end
767
768 function [F1,F2,F3,F4,G1,G2,G3,G4] = Van_Leer_Scheme( ...
769     F1i,F2i,F3i,F4i,G1i,G2i,G3i,G4i,A_x,A_y,...
770     imax, jmax, n_xface_x, n_xface_y, n_yface_x, n_yface_y, ...
771     V1_xface_L, V1_xface_R, V2_xface_L, V2_xface_R, ...
772     V3_xface_L, V3_xface_R, V4_xface_L, V4_xface_R, ...
773     V1_yface_D, V1_yface_U, V2_yface_D, V2_yface_U, ...
774     V3_yface_D, V3_yface_U, V4_yface_D, V4_yface_U)
775
776     F1 = F1i;
777     F2 = F2i;

```

```

777 F3 = F3i;
778 F4 = F4i;
779 G1 = G1i;
780 G2 = G2i;
781 G3 = G3i;
782 G4 = G4i;
783 gamma = 1.4;
784
785 % For x - face
786 for j = 1 : jmax - 1
787     for i = 1 : imax
788         rho_l = V1_xface_L(i, j);
789         rho_r = V1_xface_R(i, j);
790
791         u_l = V2_xface_L(i, j);
792         u_r = V2_xface_R(i, j);
793
794         v_l = V3_xface_L(i, j);
795         v_r = V3_xface_R(i, j);
796
797         p_l = V4_xface_L(i, j);
798         p_r = V4_xface_R(i, j);
799
800         h0_l = gamma/(gamma-1)*(p_l/rho_l) + 0.5*(u_l^2 + v_l^2);
801         h0_r = gamma/(gamma-1)*(p_r/rho_r) + 0.5*(u_r^2 + v_r^2);
802
803         a_l = sqrt(gamma*p_l/rho_l);
804         a_r = sqrt(gamma*p_r/rho_r);
805         a_avg = 0.5*(a_l + a_r);
806
807         unl = u_l*n_xface_x(i, j) + v_l*n_xface_y(i, j);
808         unr = u_r*n_xface_x(i, j) + v_r*n_xface_y(i, j);
809
810         M_l = unl/a_avg; M_r = unr/a_avg;
811
812         alpha_l = 0.5*(1+sgn(M_l));
813         alpha_r = 0.5*(1-sgn(M_r));
814
815         beta_l = -max(0, 1 - floor(abs(M_l)));
816         beta_r = -max(0, 1 - floor(abs(M_r)));
817
818         CVL_p = alpha_l*(1+beta_l)*M_l - 0.25*beta_l*(M_l+1)^2;
819         CVL_m = alpha_r*(1+beta_r)*M_r + 0.25*beta_r*(M_r - 1)^2;
820
821         D_p = 0.25*(M_l+1)^2*(2-M_l);
822         D_m = 0.25*(M_r-1)^2*(2+M_r);
823
824         D_l = alpha_l*(1+beta_l) - beta_l*D_p;
825         D_r = alpha_r*(1+beta_r) - beta_r*D_m;
826
827         fm_p = A_x(i, j)*a_avg*CVL_p*rho_l;
828         fm_m = A_x(i, j)*a_avg*CVL_m*rho_r;
829
830         F1(i, j) = fm_p + fm_m;
831         F2(i, j) = fm_p*u_l+fm_m*u_r+ A_x(i, j)*(D_l*p_l + D_r*p_r)*(n_xface_x(i, j));
832         F3(i, j) = fm_p*v_l+fm_m*v_r+ A_x(i, j)*(D_l*p_l + D_r*p_r)*(n_xface_y(i, j));
833         F4(i, j) = fm_p*h0_l + fm_m*h0_r;
834     end
835 end
836
837 % For y - face
838 for i = 1 : imax - 1
839     for j = 1 : jmax
840         rho_d = V1_yface_D(i, j);
841         rho_u = V1_yface_U(i, j);
842
843         u_d = V2_yface_D(i, j);
844         u_u = V2_yface_U(i, j);
845
846         v_d = V3_yface_D(i, j);
847         v_u = V3_yface_U(i, j);
848
849         p_d = V4_yface_D(i, j);

```

```

850     p_u = V4_yface_U(i,j);
851
852     h0_d = gamma/(gamma-1)*(p_d/rho_d) + 0.5*(u_d^2 + v_d^2);
853     h0_u = gamma/(gamma-1)*(p_u/rho_u) + 0.5*(u_u^2 + v_u^2);
854
855     a_d = sqrt(gamma*p_d/rho_d);
856     a_u = sqrt(gamma*p_u/rho_u);
857     a_avg = 0.5*(a_u + a_d);
858
859     und = u_d*n_yface_x(i,j) + v_d*n_yface_y(i,j);
860     unu = u_u*n_yface_x(i,j) + v_u*n_yface_y(i,j);
861
862     M_d = und/a_avg; M_u = unu/a_avg;
863
864     alpha_d = 0.5*(1+sgn(M_d));
865     alpha_u = 0.5*(1+sgn(M_u));
866
867     beta_d = -max(0, 1 - floor(abs(M_d)));
868     beta_u = -max(0, 1 - floor(abs(M_u)));
869
870     CVL_p = alpha_d*(1+beta_d)*M_d - 0.25*beta_d*(M_d+1)^2;
871     CVL_m = alpha_u*(1+beta_u)*M_u + 0.25*beta_u*(M_u-1)^2;
872
873     D_p = 0.25*(M_d+1)^2*(2-M_d);
874     D_m = 0.25*(M_u-1)^2*(2+M_u);
875
876     D_d = alpha_d*(1+beta_d) - beta_d*D_p;
877     D_u = alpha_u*(1+beta_u) - beta_u*D_m;
878
879     fm_p = A_y(i,j)*a_avg*CVL_p*rho_d;
880     fm_m = A_y(i,j)*a_avg*CVL_m*rho_u;
881
882     G1(i,j) = fm_p + fm_m;
883     G2(i,j) = fm_p*u_d+fm_m*u_u+ A_y(i,j)*(D_d*p_d + D_u*p_u)*(n_yface_x(i,j));
884     G3(i,j) = fm_p*v_d+fm_m*v_u+ A_y(i,j)*(D_d*p_d + D_u*p_u)*(n_yface_y(i,j));
885     G4(i,j) = fm_p*h0_d + fm_m*h0_u;
886
887     end
888 end
889
890
891 function [F1,F2,F3,F4,G1,G2,G3,G4] = AUSM_Scheme( ...
892     F1i,F2i,F3i,F4i,G1i,G2i,G3i,G4i,A_x,A_y,...
893     imax, jmax, n_xface_x, n_xface_y, n_yface_x, n_yface_y, ...
894     V1_xface_L, V1_xface_R, V2_xface_L, V2_xface_R, ...
895     V3_xface_L, V3_xface_R, V4_xface_L, V4_xface_R, ...
896     V1_yface_D, V1_yface_U, V2_yface_D, V2_yface_U, ...
897     V3_yface_D, V3_yface_U, V4_yface_D, V4_yface_U)
898
899     F1 = F1i;
900     F2 = F2i;
901     F3 = F3i;
902     F4 = F4i;
903     G1 = G1i;
904     G2 = G2i;
905     G3 = G3i;
906     G4 = G4i;
907     gamma = 1.4;
908
909     % For x - face
910     for j = 1 : jmax - 1
911         for i = 1 : imax
912             rho_l = V1_xface_L(i,j);
913             rho_r = V1_xface_R(i,j);
914
915             u_l = V2_xface_L(i,j);
916             u_r = V2_xface_R(i,j);
917
918             v_l = V3_xface_L(i,j);
919             v_r = V3_xface_R(i,j);
920
921             p_l = V4_xface_L(i,j);
922             p_r = V4_xface_R(i,j);

```

```

923     h0_l = gamma/(gamma-1)*(p_l/rho_l) +0.5*(u_l^2 + v_l^2);
924     h0_r = gamma/(gamma-1)*(p_r/rho_r) +0.5*(u_r^2 + v_r^2);
925
926     a_l = sqrt(gamma*p_l/rho_l);
927     a_r = sqrt(gamma*p_r/rho_r);
928     a_avg = 0.5*(a_l + a_r);
929
930     unl = u_l*n_xface_x(i,j) + v_l*n_xface_y(i,j);
931     unr = u_r*n_xface_x(i,j) + v_r*n_xface_y(i,j);
932
933     M_l = unl/a_avg; M_r = unr/a_avg;
934
935     alpha_l = 0.5*(1+sgn(M_l));
936     alpha_r = 0.5*(1-sgn(M_r));
937
938     beta_l = -max(0, 1 - floor(abs(M_l)));
939     beta_r = -max(0, 1 - floor(abs(M_r)));
940
941     CVL_p = alpha_l*(1+beta_l)*M_l - 0.25*beta_l*(M_l+1)^2;
942     CVL_m = alpha_r*(1+beta_r)*M_r + 0.25*beta_r*(M_r - 1)^2;
943
944     CLS_p = max(0,CVL_p+CVL_m);
945     CLS_m = min(0,CVL_p+CVL_m);
946
947     D_p = 0.25*(M_l+1)^2*(2-M_l);
948     D_m = 0.25*(M_r-1)^2*(2+M_r);
949
950     D_l = alpha_l*(1+beta_l) - beta_l*D_p;
951     D_r = alpha_r*(1+beta_r) - beta_r*D_m;
952
953     fm_p = A_x(i,j)*a_avg*CLS_p*rho_l;
954     fm_m = A_x(i,j)*a_avg*CLS_m*rho_r;
955
956     F1(i,j) = fm_p + fm_m;
957     F2(i,j) = fm_p*u_l+fm_m*u_r+ A_x(i,j)*(D_l*p_l + D_r*p_r)*(n_xface_x(i,j));
958     F3(i,j) = fm_p*v_l+fm_m*v_r+ A_x(i,j)*(D_l*p_l + D_r*p_r)*(n_xface_y(i,j));
959     F4(i,j) = fm_p*h0_l + fm_m*h0_r;
960
961 end
962
963 end
964
965 % For y - face
966 for i = 1 : imax - 1
967     for j = 1 : jmax
968         rho_d = V1_yface_D(i,j);
969         rho_u = V1_yface_U(i,j);
970
971         u_d = V2_yface_D(i,j);
972         u_u = V2_yface_U(i,j);
973
974         v_d = V3_yface_D(i,j);
975         v_u = V3_yface_U(i,j);
976
977         p_d = V4_yface_D(i,j);
978         p_u = V4_yface_U(i,j);
979
980         h0_d = gamma/(gamma-1)*(p_d/rho_d) +0.5*(u_d^2 + v_d^2);
981         h0_u = gamma/(gamma-1)*(p_u/rho_u) +0.5*(u_u^2 + v_u^2);
982
983         a_d = sqrt(gamma*p_d/rho_d);
984         a_u = sqrt(gamma*p_u/rho_u);
985         a_avg = 0.5*(a_u + a_d);
986
987         und = u_d*n_yface_x(i,j) + v_d*n_yface_y(i,j);
988         unu = u_u*n_yface_x(i,j) + v_u*n_yface_y(i,j);
989
990         M_d = und/a_avg; M_u = unu/a_avg;
991
992         alpha_d = 0.5*(1+sgn(M_d));
993         alpha_u = 0.5*(1-sgn(M_u));
994
995         beta_d = -max(0, 1 - floor(abs(M_d)));
996         beta_u = -max(0, 1 - floor(abs(M_u)));

```

```

996
997     CVL_p = alpha_d*(1+beta_d)*M_d- 0.25*beta_d*(M_d+1)^2;
998     CVL_m = alpha_u*(1+beta_u)*M_u + 0.25*beta_u*(M_u-1)^2;
999
1000     CLS_p = max(0,CVL_p+CVL_m);
1001     CLS_m = min(0,CVL_p+CVL_m);
1002
1003     D_p = 0.25*(M_d+1)^2*(2-M_d);
1004     D_m = 0.25*(M_u-1)^2*(2+M_u);
1005
1006     D_d = alpha_d*(1+beta_d) - beta_d*D_p;
1007     D_u = alpha_u*(1+beta_u) - beta_u*D_m;
1008
1009     fm_p = A_y(i,j)*a_avg*CLS_p*rho_d;
1010     fm_m = A_y(i,j)*a_avg*CLS_m*rho_u;
1011
1012     G1(i,j) = fm_p + fm_m;
1013     G2(i,j) = fm_p*u_d+fm_m*u_u+ A_y(i,j)*(D_d*p_d + D_u*p_u)*(n_yface_x(i,j));
1014     G3(i,j) = fm_p*v_d+fm_m*v_u+ A_y(i,j)*(D_d*p_d + D_u*p_u)*(n_yface_y(i,j));
1015     G4(i,j) = fm_p*h0_d + fm_m*h0_u;
1016
1017     end
1018 end
1019
1020
1021 function [Res1,Res2,Res3,Res4,dt_by_Vol,V_1,V_2,V_3,V_4,F1,F2,F3,F4,G1,G2,G3,G4] = ...
1022     computeResidual(v_inf,u_inf,p_inf,rho_inf,...
1023         V_1i,V_2i,V_3i,V_4i, ...
1024         M_xi,M_yi,...
1025         F1i,F2i,F3i,F4i,G1i,G2i,G3i,G4i,...
1026         V1_xface_Li, V1_xface_Ri, V1_yface_Ui, V1_yface_Di,...
1027         V2_xface_Li, V2_xface_Ri, V2_yface_Ui, V2_yface_Di,...
1028         V3_xface_Li, V3_xface_Ri, V3_yface_Ui, V3_yface_Di,...
1029         V4_xface_Li, V4_xface_Ri, V4_yface_Ui, V4_yface_Di,...
1030         imax,jmax,CFL, ...
1031         n_xface_x,n_xface_y,n_yface_x,n_yface_y, ...
1032         A_x,A_y,func,scheme)
1033
1034     gamma = 1.4; R = 287;
1035     V_1 = V_1i; V_2 = V_2i; V_3 = V_3i; V_4 = V_4i;
1036
1037     V1_xface_L = V1_xface_Li; V1_xface_R = V1_xface_Ri;
1038     V1_yface_U = V1_yface_Ui; V1_yface_D = V1_yface_Di;
1039     V2_xface_L = V2_xface_Li; V2_xface_R = V2_xface_Ri;
1040     V2_yface_U = V2_yface_Ui; V2_yface_D = V2_yface_Di;
1041     V3_xface_L = V3_xface_Li; V3_xface_R = V3_xface_Ri;
1042     V3_yface_U = V3_yface_Ui; V3_yface_D = V3_yface_Di;
1043     V4_xface_L = V4_xface_Li; V4_xface_R = V4_xface_Ri;
1044     V4_yface_U = V4_yface_Ui; V4_yface_D = V4_yface_Di;
1045
1046     M_x = M_xi; M_y = M_yi;
1047     F1 = F1i;
1048     F2 = F2i;
1049     F3 = F3i;
1050     F4 = F4i;
1051     G1 = G1i;
1052     G2 = G2i;
1053     G3 = G3i;
1054     G4 = G4i;
1055     %% Boundary conditions and initial values
1056
1057     % Inflow
1058     for j = 2 : jmax
1059         if M_x(1,j) >=1
1060             V_1(1,j) = rho_inf;
1061             V_2(1,j) = u_inf;
1062             V_3(1,j) = v_inf;
1063             V_4(1,j) = p_inf;
1064         elseif M_x(1,j) < 1
1065             V_1(1,j) = rho_inf;
1066             V_2(1,j) = u_inf;
1067             V_3(1,j) = v_inf;
1068             V_4(1,j) = V_4(2,j);
1069         end
1070     end

```

```

1069     end
1070
1071     % Outflow
1072     for j = 2 : jmax
1073         if M_x(imax+1,j) >=1
1074             V_1(imax+1,j) = V_1(imax,j);
1075             V_2(imax+1,j) = V_2(imax,j);
1076             V_3(imax+1,j) = V_3(imax,j);
1077             V_4(imax+1,j) = V_4(imax,j);
1078         elseif M_x(imax+1,j) < 1
1079             V_1(imax+1,j) = V_1(imax,j);
1080             V_2(imax+1,j) = V_2(imax,j);
1081             V_3(imax+1,j) = V_3(imax,j);
1082             V_4(imax+1,j) = p_inf;
1083         end
1084     end
1085
1086     % Bottom Wall
1087     G1(:,1) = 0;
1088     V_1(:,1) = V_1(:,2);
1089     V_4(:,1) = V_4(:,2);
1090     U_contrav_lower = V_2(2:imax,2).*n_yface_x(:,1) + V_3(2:imax,2).*n_yface_y(:,1);
1091     V_2(2:imax,1) = V_2(2:imax, 2) - 2*U_contrav_lower.*n_yface_x(:,1);
1092     V_3(2:imax,1) = V_3(2:imax, 2) - 2*U_contrav_lower.*n_yface_y(:,1);
1093
1094     % Top Wall
1095     G1(:,jmax) = 0;
1096     V_1(:,jmax+1) = V_1(:,jmax);
1097     V_4(:,jmax+1) = V_4(:,jmax);
1098     U_contrav_upper = V_2(2:imax,jmax).*n_yface_x(:,jmax) +
1099     ↪ V_3(2:imax,jmax).*n_yface_y(:,jmax);
1100     V_2(2:imax,jmax+1) = V_2(2:imax, jmax) - 2*U_contrav_upper.*n_yface_x(:,jmax);
1101     V_3(2:imax,jmax+1) = V_3(2:imax, jmax) - 2*U_contrav_upper.*n_yface_y(:,jmax);
1102
1103     % Bottom-left corner
1104     V_1(1,1) = V_1(2,2);
1105     V_2(1,1) = V_2(2,2);
1106     V_3(1,1) = V_3(2,2);
1107     V_4(1,1) = V_4(2,2);
1108
1109     % Bottom-right corner
1110     V_1(imax+1,1) = V_1(imax,2);
1111     V_2(imax+1,1) = V_2(imax,2);
1112     V_3(imax+1,1) = V_3(imax,2);
1113     V_4(imax+1,1) = V_4(imax,2);
1114
1115     % Top-left corner
1116     V_1(1,jmax+1) = V_1(2,jmax);
1117     V_2(1,jmax+1) = V_2(2,jmax);
1118     V_3(1,jmax+1) = V_3(2,jmax);
1119     V_4(1,jmax+1) = V_4(2,jmax);
1120
1121     % Top-right corner
1122     V_1(imax+1,jmax+1) = V_1(imax,jmax);
1123     V_2(imax+1,jmax+1) = V_2(imax,jmax);
1124     V_3(imax+1,jmax+1) = V_3(imax,jmax);
1125     V_4(imax+1,jmax+1) = V_4(imax,jmax);
1126
1127     %% Interface Reconstruction
1128
1129     for j = 2 : jmax
1130         for i = 2 : imax
1131
1132             dV_1_p = V_1(i+1,j) - V_1(i,j);
1133             dV_1_m = V_1(i,j) - V_1(i-1,j);
1134             V1_xface_L(i,j-1) = V_1(i,j) + 0.5*limiter(dV_1_p,dV_1_m,func);
1135             V1_xface_R(i-1,j-1) = V_1(i,j) - 0.5*limiter(dV_1_p,dV_1_m,func);
1136
1137             dV_2_p = V_2(i+1,j) - V_2(i,j);
1138             dV_2_m = V_2(i,j) - V_2(i-1,j);
1139             V2_xface_L(i,j-1) = V_2(i,j) + 0.5*limiter(dV_2_p,dV_2_m,func);
1140             V2_xface_R(i-1,j-1) = V_2(i,j) - 0.5*limiter(dV_2_p,dV_2_m,func);

```

```

1141
1142     dV_3_p = V_3(i+1,j) - V_3(i,j);
1143     dV_3_m = V_3(i,j) - V_3(i-1,j);
1144     V3_xface_L(i,j-1) = V_3(i,j) + 0.5*limiter(dV_3_p,dV_3_m,func);
1145     V3_xface_R(i-1,j-1) = V_3(i,j) - 0.5*limiter(dV_3_p,dV_3_m,func);
1146
1147     dV_4_p = V_4(i+1,j) - V_4(i,j);
1148     dV_4_m = V_4(i,j) - V_4(i-1,j);
1149     V4_xface_L(i,j-1) = V_4(i,j) + 0.5*limiter(dV_4_p,dV_4_m,func);
1150     V4_xface_R(i-1,j-1) = V_4(i,j) - 0.5*limiter(dV_4_p,dV_4_m,func);
1151
1152
1153     end
1154     V1_xface_L(1,j-1) =V_1(1,j); V1_xface_R(imax,j-1) =V_1(imax+1,j);
1155     V2_xface_L(1,j-1) =V_2(1,j); V2_xface_R(imax,j-1) =V_2(imax+1,j);
1156     V3_xface_L(1,j-1) =V_3(1,j); V3_xface_R(imax,j-1) =V_3(imax+1,j);
1157     V4_xface_L(1,j-1) =V_4(1,j); V4_xface_R(imax,j-1) =V_4(imax+1,j);
1158
1159 end
1160
1161 % For y-faces
1162 for i = 2 : imax
1163     for j = 2 : jmax
1164
1165         dV_1_p = V_1(i,j+1) - V_1(i,j);
1166         dV_1_m = V_1(i,j) - V_1(i,j-1);
1167         V1_yface_D(i-1,j) = V_1(i,j) + 0.5*limiter(dV_1_p,dV_1_m,func);
1168         V1_yface_U(i-1,j-1) = V_1(i,j) - 0.5*limiter(dV_1_p,dV_1_m,func);
1169
1170         dV_2_p = V_2(i,j+1) - V_2(i,j);
1171         dV_2_m = V_2(i,j) - V_2(i,j-1);
1172         V2_yface_D(i-1,j) = V_2(i,j) + 0.5*limiter(dV_2_p,dV_2_m,func);
1173         V2_yface_U(i-1,j-1) = V_2(i,j) - 0.5*limiter(dV_2_p,dV_2_m,func);
1174
1175         dV_3_p = V_3(i,j+1) - V_3(i,j);
1176         dV_3_m = V_3(i,j) - V_3(i,j-1);
1177         V3_yface_D(i-1,j) = V_3(i,j) + 0.5*limiter(dV_3_p,dV_3_m,func);
1178         V3_yface_U(i-1,j-1) = V_3(i,j) - 0.5*limiter(dV_3_p,dV_3_m,func);
1179         dV_4_p = V_4(i,j+1) - V_4(i,j);
1180         dV_4_m = V_4(i,j) - V_4(i,j-1);
1181         V4_yface_D(i-1,j) = V_4(i,j) + 0.5*limiter(dV_4_p,dV_4_m,func);
1182         V4_yface_U(i-1,j-1) = V_4(i,j) - 0.5*limiter(dV_4_p,dV_4_m,func);
1183
1184     end
1185     V1_yface_D(i-1,1) =V_1(i,1); V1_yface_U(i-1,jmax) =V_1(i,jmax+1);
1186     V2_yface_D(i-1,1) =V_2(i,1); V2_yface_U(i-1,jmax) =V_2(i,jmax+1);
1187     V3_yface_D(i-1,1) =V_3(i,1); V3_yface_U(i-1,jmax) =V_3(i,jmax+1);
1188     V4_yface_D(i-1,1) =V_4(i,1); V4_yface_U(i-1,jmax) =V_4(i,jmax+1);
1189
1190 end
1191
1192 %% Storing Face Fluxes
1193
1194 if scheme == 0
1195     [F1,F2,F3,F4,G1,G2,G3,G4] = Roe_Scheme( ...
1196         F1,F2,F3,F4,G1,G2,G3,G4,A_x,A_y,...
1197         imax, jmax, n_xface_x, n_xface_y, n_yface_x, n_yface_y, ...
1198         V1_xface_L, V1_xface_R, V2_xface_L, V2_xface_R, ...
1199         V3_xface_L, V3_xface_R, V4_xface_L, V4_xface_R, ...
1200         V1_yface_D, V1_yface_U, V2_yface_D, V2_yface_U, ...
1201         V3_yface_D, V3_yface_U, V4_yface_D, V4_yface_U);
1202 elseif scheme == 1
1203     [F1,F2,F3,F4,G1,G2,G3,G4] = Van_Leer_Scheme( ...
1204         F1,F2,F3,F4,G1,G2,G3,G4,A_x,A_y,...
1205         imax, jmax, n_xface_x, n_xface_y, n_yface_x, n_yface_y, ...
1206         V1_xface_L, V1_xface_R, V2_xface_L, V2_xface_R, ...
1207         V3_xface_L, V3_xface_R, V4_xface_L, V4_xface_R, ...
1208         V1_yface_D, V1_yface_U, V2_yface_D, V2_yface_U, ...
1209         V3_yface_D, V3_yface_U, V4_yface_D, V4_yface_U);
1210 elseif scheme == 2
1211     [F1,F2,F3,F4,G1,G2,G3,G4] = AUSM_Scheme( ...
1212         F1,F2,F3,F4,G1,G2,G3,G4,A_x,A_y,...
1213         imax, jmax, n_xface_x, n_xface_y, n_yface_x, n_yface_y, ...

```

```

1214         V1_xface_L, V1_xface_R, V2_xface_L, V2_xface_R, ...
1215         V3_xface_L, V3_xface_R, V4_xface_L, V4_xface_R, ...
1216         V1_yface_D, V1_yface_U, V2_yface_D, V2_yface_U, ...
1217         V3_yface_D, V3_yface_U, V4_yface_D, V4_yface_U);
1218     end
1219
1220     %% Residual Calculation
1221
1222     Res1 = zeros(imax-1, jmax-1);
1223     Res2 = zeros(imax-1, jmax-1);
1224     Res3 = zeros(imax-1, jmax-1);
1225     Res4 = zeros(imax-1, jmax-1);
1226
1227     for i = 2:imax
1228         for j = 2:jmax
1229             Res1(i-1,j-1) = F1(i,j-1) - F1(i-1,j-1) + G1(i-1,j) - G1(i-1,j-1);
1230             Res2(i-1,j-1) = F2(i,j-1) - F2(i-1,j-1) + G2(i-1,j) - G2(i-1,j-1);
1231             Res3(i-1,j-1) = F3(i,j-1) - F3(i-1,j-1) + G3(i-1,j) - G3(i-1,j-1);
1232             Res4(i-1,j-1) = F4(i,j-1) - F4(i-1,j-1) + G4(i-1,j) - G4(i-1,j-1);
1233         end
1234     end
1235
1236     %% Time Step Estimation
1237
1238     dt_by_Vol = zeros(imax-1, jmax-1);
1239
1240     for j = 2:jmax
1241         for i = 2:imax
1242             Vn1 = V_2(i,j) * n_xface_x(i,j-1) + V_3(i,j) * n_xface_y(i,j-1); % right
1243             Vn2 = V_2(i,j) * n_yface_x(i-1,j) + V_3(i,j) * n_yface_y(i-1,j); % up
1244             Vn3 = V_2(i,j) * n_xface_x(i-1,j-1) + V_3(i,j) * n_xface_y(i-1,j-1); %
1245                 ↪ left
1246             Vn4 = V_2(i,j) * n_yface_x(i-1,j-1) + V_3(i,j) * n_yface_y(i-1,j-1); %
1247                 ↪ down
1248
1249             a = sqrt(gamma*V_4(i,j)/V_1(i,j));
1250             lambda1 = abs(Vn1)+a;
1251             lambda2 = abs(Vn2)+a;
1252             lambda3 = abs(Vn3)+a;
1253             lambda4 = abs(Vn4)+a;
1254             temp = A_x(i,j-1)*lambda1 + A_y(i-1,j)*lambda2 + A_x(i-1,j-1)*lambda3...
1255                 + A_y(i-1,j-1)*lambda4;
1256             dt_by_Vol(i-1,j-1) = CFL/temp;
1257         end
1258     end
1259 end

```